

Classify Traffic Rather Than Flow: Versatile Multi-Flow Encrypted Traffic Classification With Flow Clustering

Zihan Chen^{ID}, *Member, IEEE*, Guang Cheng^{ID}, *Member, IEEE*, Zijun Wei, *Student Member, IEEE*,
Dandan Niu, *Student Member, IEEE*, and Nan Fu, *Student Member, IEEE*

Abstract—Encrypted Traffic Classification (ETC) can provide necessary information support for network management and security. The state-of-the-art ETC methods take a single flow as the unit and only use sequence features based on in-flow relationships. In an actual network, one-time access to an application will generate multiple flows. Taking a single flow as the classification unit will produce many repeated and potentially erroneous results, which dramatically reduces the classification efficiency and prevents the results from being used for effective network management and security. In this paper, we propose a multi-flow ETC method. Since multiple flows generated by an application cannot be directly bound in a complex multi-application scenario, we first cluster the encrypted traffic to acquire flow bunches through the proposed Time Sequential Hierarchical Clustering with Sliding Windows (TSHC-SW) algorithm. Then, based on the flow bunches, we propose five different multi-flow classification schemas that can realize multi-flow classification effectively with model-independent. Open-world experiments show that our method is versatile in that it can pursue classification accuracy, speed, or sample covering rate, respectively, according to the actual demand and network environment constraints. In flow clustering, we achieve 95% adjusted Rand Index and 98% purity. In the multi-flow classification, we have over 99% F1-score, 79% prediction time saving, and 5% sample covering rate increasing, which is far superior to the state-of-the-art single-flow methods.

Index Terms—Multi-flow encrypted traffic classification, inter-flow relationship, flow clustering and flow bunch, TSHC-SW, open-world environment.

I. INTRODUCTION

ENCRYPTED traffic has become the main form of data transmission under the Internet of Everything. Currently, more than 95% of the traffic is encrypted traffic [1]. Encryption covers the load information in the original plaintext traffic, making the matching method based on Deep Packet Inspection (DPI) invalid [2]. The unavailability of information related to traffic on the network side brings significant challenges to

network management, information authentication, and network security of ISP as well as security enterprises. This situation makes encrypted traffic analysis, especially Encrypted Traffic Classification (ETC), which can effectively obtain the current traffic, become a hot and essential research direction. Moreover, ETC makes up for the shortcomings of log analysis based on the end system and has the advantages of large-scale, near real-time, transparent, and cross-temporal analysis. For network management, ISPs can synchronize the analysis of millions of users on the backbone border gateway through ETC to achieve billion-level traffic acceleration to ensure user Quality of Experience (QoE). For network security, ETC can identify and block network attacks even before they occur. ETC refers to classifying encrypted traffic into specified known services or applications by extracting features not covered by the encryption. As the smallest independent unit for users to interact on the Internet, the application is highly bound to the enterprise. Therefore, the application classification of encrypted traffic can enable ISPs to secure the transmission of specific applications of specific enterprises, thus obtaining numerous benefits. This process is also the basis for fine-grained application behavior auditing (ISPs must first know which application the traffic belongs to before further analyzing it).

Initially, academia and industry tended to use residual plaintext information, such as TLS SNI or plaintext DNS on ETC. Although encrypted DNS (e.g., DoH, DoT) is not widely deployed, and ESNI or ECH is far from being adopted, these technologies, as well as the TLS-1.3 protocol, will completely invalidate such methods. The complete absence of deterministic residual plaintext information makes Artificial Intelligence (AI) methods widely used in ETC. As a typical supervised learning problem, AI methods applied include traditional machine learning [3], [4], [5], ensemble learning [6], [7] and deep learning [8], [9], [10], [11], [12], [13], [14]. The existing ETC methods mostly take the flow [14] or the packet sequence in the flow as the classification unit and extract the statistical features [3], the residual plaintext features [4], the inter-arrival time sequence features [15] and the length sequence features [13] for classification. These methods are mainly divided into two research paths, encrypted traffic classifier optimization [16] or encrypted traffic feature engineering [17]. In addition, there is also some research on the clustering of encrypted traffic. Unknown applications or services are mainly discovered through feature clustering [18]. However,

Manuscript received 22 February 2023; revised 6 July 2023 and 7 September 2023; accepted 1 October 2023. Date of publication 9 October 2023; date of current version 15 April 2024. This work is supported by the General Program of the National Natural Science Foundation of China under Grant No. 62172093 and the Joint Funds of the National Natural Science Foundation of China under Grant No. U22B2025. The associate editor coordinating the review of this article and approving it for publication was T. Inoue. (Corresponding author: Guang Cheng.)

The authors are with the School of Cyber Science and Engineering, Southeast University, Nanjing 210096, China (e-mail: chengguang@seu.edu.cn; gcheng@njnet.edu.cn).

Digital Object Identifier 10.1109/TNSM.2023.3322861

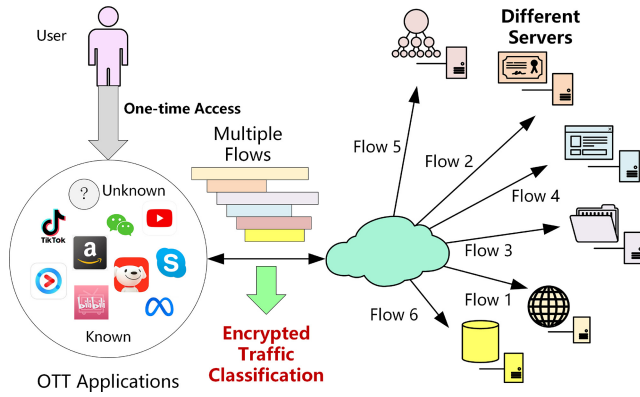


Fig. 1. Multi-resource request diagram of Over-The-Top (OTT) applications.

it is worth noting that since the flow is the atomized efficient unit in network management and analysis, the existing research on encrypted traffic clustering also takes flow as the basic unit.

However, the above methods ignore an essential phenomenon of network application transmission – an application or Web page providing services, or even some system control behavior, will generate multiple flows in the process of one-time access [19], as shown in Figure 1. Existing research on ETC or feature clustering takes a single flow as the processing unit. However, multiple network flows from a single access are correlated with each other [2]. Hence, in the actual network environment, if each flow is classified, the following problems may exist: (1) Multiple flows generated by one application, even in one-time access, have different importance and significance. The classification results of each flow may be different in terms of category and confidence, leading to a consequential error or even unavailability of the results. (2) The number of flows generated by the access is enormous, but multiple flows all point to the same application. The repeated classification wastes computing resources greatly, and the throughput is reduced exponentially. (3) The information of a single flow generated by the application may be limited, and the classification results with high confidence cannot be obtained. (4) Application of the external hyperlink and reusing the same function or open Web service of a vendor will result in a partial diversion of reference or mutual confusion, leading to misclassification. Due to the above problems, the current research on ETC is still challenging to apply to the actual network environment, especially with poor effects in the open-world online scenario.

A few existing pieces of research also use multiple flows to encrypt traffic classification [5], [20]. It uses multiple flows in a TLS/TCP session, that is, all the flows that are transmitted between two communicators over a period of time (usually during the life of the session, that is, before it expires). However, the behavior of multiple flows in a session is not necessarily related, and the ubiquitous randomized flow interspersion brings noticeable conceptual drift to the classification. In addition, the traffic classification of the entire session is not interpretable, and it is not easy to give a normalized label to the traffic of the entire session. In addition, from the classifier's input, it is still a single flow classification in essence.

Therefore, this paper focuses on classifying encrypted traffic for multiple flows generated by one-time access to network applications or services. Our method is fundamentally different from the existing classification based on multiple flows in a session. First, we discuss in depth the pattern of network application or service data transmission to further embody the significance of multi-flow classification. Then, based on the theory of information entropy and information disclosure, we analyze the in-flow and inter-flow relationships contained in the one-time access. It provides a theoretical basis for multi-flow classification. Afterward, we put forward the concept of flow bunch and formalized the flow clustering and multi-flow classification of encrypted traffic problems. On this basis, aiming at the problem of construct flow bunch, which is the basis of multi-flow classification, we propose the Time Sequential Hierarchical Clustering with Sliding Windows (TSHC-SW) algorithm to realize the clustering of encrypted traffic. Further, we propose five demand-oriented multi-flow classification schemas for encrypted traffic, and they are based on three training modes. In order to measure different schemas, we propose the Sample Covering Rate (SCR) metric, which represents the proportion of flow bunch results that can be classified in an actual network environment. Different multi-flow classification schemas can be selected in pursuit of classification efficiency, accuracy, or applicability. We finally achieved versatility for multi-flow ETC combined with the choosing mode for flow.

Our main contributions are summarized as follows:

- We introduce multi-flow classification into ETC for the first time and deeply study the generation relationship during one-time access of an application. Based on this, the in-flow and inter-flow relationships are formally analyzed, laying a theoretical foundation for multi-flow classification.
- We aimed at the problem that the packets cannot be directly formed to all the flows in one-time access of an application. To solve this problem, we put forward the concept of a flow bunch and propose the TSHC-SW clustering method to realize the binding of multiple flows.
- We propose five kinds of demand-oriented multi-flow classification optimization schemas for encrypted traffic. It consists of three different training modes, which are model-independent. We propose a new classification metric, SCR, which can effectively describe the actual network applicability of the schemas. Each schema has its own specialties.
- We collected and published two new encrypted traffic datasets in the actual CERNET environment. The datasets contain two dimensions of labels, which can be used to study ETC and encrypted flow clustering. We conducted detailed experiments in flow clustering and multi-flow classification with state-of-the-art models. It proves that multi-flow classification is versatile and far outperforms single-flow classification.

The rest of the paper is shown as follows. Section II introduces the existing research on ETC and clustering. In Section III, we introduced encrypted application data transmission mode as well as the relationship in and between flows.

TABLE I
ALL THE ACRONYMS USED IN THIS PAPER

Acronyms	Full name	Acronyms	Full name	Acronyms	Full name
ETC	Encrypted Traffic Classification	TSHC-SW	Time Sequential Hierarchical Clustering with Sliding Windows	CERNET	China Education and Research Network
RF	Random Forest	ISP	Internet Service Providers	AI	Artificial Intelligence
SCR	Sample Covering Rate	DPI	Deep Packet Inspection	DFI	Deep Flow Inspection
SVM	Support Vector Machine	CNN	Convolutional Neural Networks	LSTM	Long Short Term Memory
CapsNet	Capsule Neural Networks	d-CT	d-Candidate Together	PDU	Protocol Data Unit
TLS	Transport Layer Security	QoS	Quality of Service	NAT	Network Address Translation
HTTP	Hyper Text Transfer Protocol	MTU	Maximum Transmission Unit	HTTPS	HTTP over TLS
IP	Internet Protocol	UDP	User Datagram Protocol	QoE	Quality of Experience
RI	Rand Index	ARI	Adjusted RI	HF	wHole Flow
SF	Single Flow	EF	Each Flow	d-CV	d-Candidate Voting
AMIC	Adjusted Mutual Information Coefficient	LSCDL	Length-Sensitive Composite Deep Learning		

Then, we formally defined the research problems of this paper. Section IV presents the flow clustering method and multi-flow classification schemas. In Section V, we conduct experiments from three aspects to demonstrate the validity of our method. Finally, we summarize our research and discuss the future research directions in Section VI. All the acronyms used in this paper are shown in Table I.

II. RELATED WORK

Existing researches mainly fall into two categories. One is to classify known services or applications, which is called ETC. The other is to discover and classify unknown services or applications, which mainly involves a clustering algorithm, summarized as encrypted traffic clustering in this paper.

A. Encrypted Traffic Classification

As a typical supervised learning problem, the two most critical elements in the supervised learning problem are feature and classifier. The initial research on ETC mainly focuses on classifier optimization, uses statistical features of flow granularity, and introduces traditional machine learning methods such as decision tree [3], and SVM [5]. However, the accuracy of traditional machine learning classification methods needs to be improved, and their performance in large-scale traffic and multi-category classification could be more satisfactory. Therefore, ensemble learning methods are gradually used in ETC. The most representative ones are the ensemble Markov chain model WENC [6] and the well-known random forest model [7].

However, both traditional machine learning and ensemble learning are faced with the problem of poor generalization ability and the requirement to specify features for classification in advance. In deep learning, the neural network is used as the classification base, which has good generalization ability and end-to-end learning characteristics, which is also known as automatic feature selection. Raw data is directly input into the neural network to realize classification. Therefore, deep learning methods are widely used in ETC, representative of which are basic models such as CNN [8], LSTM [9], Text-CNN [10], and CapsNet [11]. There are also some mixed models such as STNN (LSTM + 3D-CNN) [12] and LSCDL (bi-GRU + CapsNet/LSTM) [14]. However, the

deep learning method also has some defects, such as the lack of the interpretability of the model and the weakness in high-dimensional optimization of features [14] as well as ignoring of structural features [20]. Similar to natural language, traffic data will be cut into different PDUs due to the network protocol stack, and high-dimensional features will be lost when the original traffic data is inputted in one go, which would lead to the suboptimal. In addition, encrypted traffic, unlike images, is structured data containing security protocols and network protocol stacks, and such features will be erased when the original traffic is directly input. Hence, there spawns a large number of studies on encrypted traffic feature engineering and some interpretable artificial intelligence studies [21].

In the research of encrypted traffic feature engineering, since the packet sequence is a data sequence with time intervals, some studies start from the time series features, such as the Flowpic in the general domain [22] and time transform domain features [15].

However, more studies consider that the packet interval is not a stable feature and will be affected by the fluctuation of the network environment. Therefore, more attention is paid to the Markov properties among packets, from TLS message type state transition features [17], [23] to length sequence features [16], [24], [25], [26].

Although the packet length sequence is related to the overall application data length, taking the packet as the granularity also ignores the high-dimensional feature optimization. Therefore, there are GraphDApp model [27] using graphic packet length sequence as features, LSCDL (our previous work) [13], [14] using multi-PDU length sequence as features, and path signature features to depict packet length sequences [28]. These works are summarized as Table II.

However, all of these studies use a single flow as a unit. In the actual network environment, the traffic of an application or a service consists of multiple flows. Therefore, taking the traffic of access as a unit for analysis will have better accuracy [29]. In addition, in the scenario of big data, the volume of traffic is large, and the efficiency of a single flow classification cannot be guaranteed [30].

Very few studies consider the correlation between multiple flows and use multiple flows as the classification unit, such as using the flow triples (destination IP, destination port, protocol) to form the Bag of Flows (BoF) for further classification [31].

TABLE II
REPRESENTATIVE WORKS ON ETC (INCLUDING BOTH CLASSIFIER OPTIMIZATION AND FEATURE ENGINEERING, SORTED BY YEAR)

Works	Year	Classification Features	Classifier	Advantages
[17]	2014	FoSM	N/A	the first study of the TLS message type Markov relation
[3]	2016	flow-level statistical features	C4.5	the first systematic study of ETC
[23]	2017	SoB	bisecting K-means	introducing the length of the TLS Application Data to ETC
[5]	2017	packet/flow statistical features	SVM	using multiple granularity features
[6]	2017	TLS handshake message type and length	WENC	using correlated TLS features
[7]	2017	flow-level statistical feature	RF	using ensemble learning
[8]	2017	automatic feature selection	1D-CNN	utilizing the deep learning end-to-end specialty in ETC
[24]	2018	LaFFT	LR/RF	using DFT transformed packet length as features
[25]	2018	MaMPF	GBTs/RF	using power law distribution to fit packet length-number relationship
[16]	2019	packet length sequence	FS-Net	introducing representation learning with packet length sequence
[9]	2019	automatic feature selection	LSTM/HAN	taking advantage of RNN in sequence expression
[10]	2019	automatic feature selection	Text-CNN	using Text-CNN to better express sequential relationships
[11]	2019	automatic feature selection	CapsNet	using CapsNet to extract higher level features in traffic
[12]	2019	automatic feature selection	STNN	creating a separate composite classifier for each category
[22]	2019/2021	FlowPic	CNN	utilizing length and IAT, which has some interpretability
[13], [14]	2020/2021	multi-PDU length sequence	LS-CapsNet LS-LSTM	using multi-PDU of different protocol stacks as the input unit
[15]	2020	CWT/CQT/WVD	SVM/kNN/C4.5	using transform domain timing features
[26]	2020	TLS length sequence and application signature	App-Net	using CNN and LSTM to handling features separately
[27]	2021	TIG	GraphDApp	using graphic burst length features to classify decentralized applications
[28]	2022	Path signature feature	RF	using path signature to better describe the length state transferring

However, in most cases, BoF is just a bag of flows of different users or at different times accessing the same port on the same server (and this port is likely to be a generic port, such as 443). These flows are not correlated in behavior, and the results are challenging to support decision-making. Another research [32] uses target flow and its contextual flows with the same device IP as input. Then it classifies the flows into different applications that overlap with the target bidirectional flow duration together. However, in the actual network environment, the existence of concurrent behavior and application long flow (mostly the application session maintenance flow) will cause the failure of this method in constructing context inputs. Therefore, this paper selects multiple flows in one-time access as the classification unit.

B. Encrypted Traffic Clustering

The premise of multi-flow classification is that multiple flows belong to one access, which involves the study of encrypted traffic clustering. Encrypted traffic clustering is an unsupervised process that is based on the features of encrypted traffic. Most existing research on encrypted traffic clustering aims to classify the traffic with similar features into a service or application and then determine whether it belongs to a known one or a new one by other identification methods.

The representative clustering methods include Levenshtein distance calculation [33] in encrypted video traffic clustering, CNN combined with K-means [34] in unknown application mining from traffic, agglomerative clustering [35] as guidance for classification, and Enhanced Rough K-Means algorithm (ERKM) [36] to aggregate flows to improve QoS mapping efficiency. These works are summarized as Table III.

Aiming at the clustering of the encrypted flow, Mateless et al. [19], used K-means and other algorithms for flow clustering. This proves that it is feasible and accurate to cluster the flow of a network entity.

TABLE III
CURRENT WORKS ON ENCRYPTED TRAFFIC CLUSTERING

Works	Clustering paths	Specialties
[33]	Levenshtein distance calculation	encrypted video traffic clustering
[34]	K-means	unknown application mining
[35]	agglomerative clustering	use clustering as guidance for classification
[36]	enhanced rough K-means	QoS mapping efficiency improving
[19]	K-means	network entity flow clustering

III. PRELIMINARIES

A. Application Data Transmission Pattern

The complexity of applications leads to the diversification of resources. Taking HTTPS Web as an example, a single Web access will generate many requests and responses for data. That is, an application can generate multiple application data transfers in its use (if two-way data is transferred simultaneously, it can be called interaction). Data are carried by network traffic, and the basic unit of network traffic is the flow [16], which is formed by the piecewise decoupling mechanism [37].

In most cases, single access to an application will result in multiple transfers of application data, that is, multiple flows. These flows have two modes, the first is the continuation mode, and the second is the concurrent mode. The continuation mode means that the subsequent flow is generated after the end of a flow with the same belonging. The concurrent mode means that two flows do not have an explicit membership relationship with each other but simultaneously serve the same access.

In terms of protocols, multiple flows may use similar or completely different protocols. Because one access may interact with multiple devices, the purpose of different flows may be different, resulting in different protocols.

It is worth noting that the multi-flow classification in this paper does not consider the problem that a source IP may

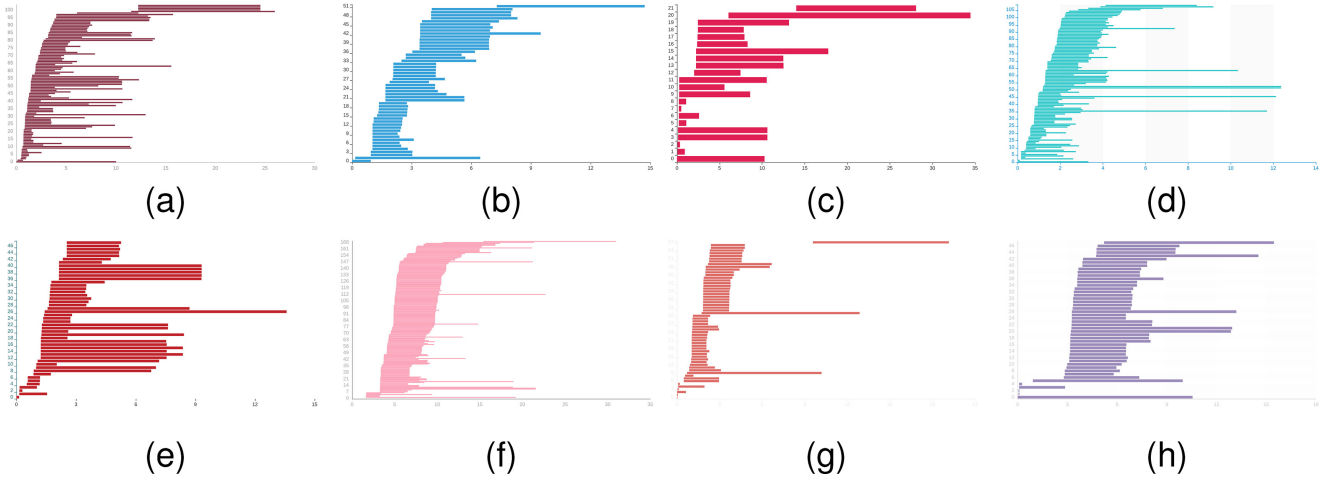


Fig. 2. Relative duration distribution of each flow in one representative access of eight applications: (a) Sohu, (b) Douban, (c) Bilibili, (d) Sina, (e) Netease, (f) JD, (g) Hupu, (h) Youku.

represent multiple network entities in the NAT environment, and the identification of multiple network entities after the same IP has been discussed in existing studies [19].

We selected eight typical applications, all of which are within the dataset proposed in this paper. According to the statistical results of the captured data, it can be found that the time distribution of multiple flows at a time is shown in Figure 2. The abscissa of each subgraph is the relative arrival time concerning the start time of the first flow, while the ordinate is the number of the flow, starting with 1.

As seen from the figure, no matter what kind of application, many flows will be generated in the process of one-time access, and these flows are almost all continuous in the time distribution. The result is consistent with our analysis, and this phenomenon indicates the necessity of flow clustering. The real application usage information can be obtained only by classifying all the flows of the access. Otherwise, many same application classification results will be obtained, which may contain many errors. Meanwhile, there are apparent differences in the timestamp distribution of multiple flows generated by one access in different applications, which also provides essential feasibility support for multi-flow classification.

B. Access Level Encrypted Traffic Length Sequences

ETC relies on features, and better features will provide more information disclosure, thus raising the upper limit of classification effectiveness [38]. Hence, we should explore the residual features of encrypted traffic as much as possible to make the relative information gain greater to improve the theoretical classification effect of encrypted traffic. The length sequence feature is the most effective in ETC, so this paper selects the packet length sequence as the basic feature.

It is worth noting that information gain is the theoretical upper limit. In practice, using this feature does not mean that we can fully utilize this feature, and this upper limit needs to be approximated by the classification model.

Each flow has its sequence of packet lengths for multiple flows generated by a single access. The encrypted traffic packet

length sequence is a structured discrete value sequence in which there is a Markov relationship between each value, which is referred to as an in-flow relationship. The flows within a single access are correlated, so there is also a certain correlation between the packet length sequences of multiple flows. This correlation comes from the resource correlation caused by the application data transmission mode, which is called the inter-flow relationship.

The theoretical advantage of multi-flow classification is that it can use the additional feature space of multi-flow to reduce uncertainty and improve the classification accuracy's upper limit without changing the sample entropy and the original feature condition entropy. Formally, suppose X is the sample space labeled by category, and Y is the feature space of single-flow classification. It is equivalent to more than an extra encrypted traffic feature space Z , making $H(X|Y)$ becoming $H(X|YZ)$ (H is the conditional entropy). The formula is:

$$\begin{aligned}
 H(X|YZ) &= \sum_{y,z \in Y,Z} \Pr(yz) H(X|(Y=y, Z=z)) \\
 &= - \sum_{y,z \in Y,Z} \Pr(y) \Pr(z) \\
 &\quad \sum_{x \in X} \Pr(x|y) \Pr(x|z) \log_2 \Pr(x|y) \Pr(x|z)
 \end{aligned} \tag{1}$$

Therefore, we should also select encrypted traffic packet length sequences from multiple flows and combine them to form new features, which is the theoretical source of feature selection for multi-flow classification.

C. Problem Definition

Two problems need to be considered in this paper: the clustering of encrypted traffic and the multi-flow classification of encrypted traffic. The clustering of encrypted traffic is a prerequisite for the multi-flow ETC.

Firstly, we formally describe the problem of encrypted flow clustering. To realize the clustering, we propose the concept of **flow bunch**. A flow bunch refers to a collection of flows

formed by flow clustering with certain features around a single device at one access.

Therefore, the encrypted flow clustering problem is a process of combining multiple encrypted flows to form a flow bunch based on certain features. Assume that the encrypted traffic to be analyzed is E , and a total of N encrypted flow inputs are included. These flows need to be clustered. The actual application access to which they belong is T , and there are A pieces of access in total. Then, all the flows generated by the i -th access are $t_i, t_i \in T, i \leq A, \forall p \in t_i, p \in E$.

However, in the real world, we do not know the real number of access, and we also do not know how many flows there are for each access. Assume that the k -th flow in all the N flows can be expressed as $p_k = [f_1^{(k)}, f_2^{(k)}, \dots, f_m^{(k)}]$, which m is the total number of clustering features, $f_i^{(k)}$ on behalf of the i -th feature. Then encrypted flow clustering's goal is to build a clustering model, $\theta(p_1, \dots, p_N)$ to obtain $\tilde{A}$ bunches which is expected to be equal to A , and each clustering produces a bunch $b_i = \tilde{t}_i$ as far as possible the same as the t_i .

Based on the flow bunches acquired by encrypted flow clustering, the problem of multi-flow ETC is to classify flow bunches based on in-flow length sequence features and inter-flow length sequence features. Suppose that there are currently M flow bunches to be classified, and there are C different application categories, the k -th flow bunch $b_k = [x_1^{(k)}, x_2^{(k)}, \dots, x_u^{(k)}]$, and the j -th sample flow in the flow bunch $x_j = [l_1^{(j)}, l_2^{(j)}, \dots, l_r^{(j)}]$, where $l_q^{(j)}$ represents the length of q -th packet in the flow.

Hence, this paper's ETC goal is to build a classification model $\phi(w_k)$ and effectively combine the above features of multiple flows in one flow bunch to get a forecast label $\tilde{S}_k$ which is expected to be the same as the real label S_k .

It is worth noting that since the ETC in this paper is access-oriented, the concurrent flows of a given app must execute one and only one transaction. Hence, we do not need to consider the problem of different flows acting on different functions, as different functions that users do will be different pieces of access.

IV. MULTI-FLOW ENCRYPTED TRAFFIC CLASSIFICATION

In order to realize multi-flow classification, we first cluster multiple flows generated by one access. After clustering multiple flows, we further study the multi-flow-oriented ETC approaches.

A. Flow Clustering

Since this paper is focused on multi-flow classification, we first need to clarify what "multi-flow" is.

From the perspective of traffic generation, multi-flow is defined as a collection of all flows generated by an application during single access. If the same application is accessed multiple times in a row, the flows should not belong to the same multi-flow classification unit (i.e., flow bunch). In other words, multi-flow classification is access-oriented and takes access as the minimum unit. It is worth noting that both TCP

and UDP may be used in a single access, depending on each resource's transmission requirements and configurations.

Theoretically, all flows in single access belong to the same source IP address. However, in the actual network environment, we find that some applications use both IPv4 and IPv6 in a single access. If the source IP address is used only to divide access devices (In actual networks, traffic collection points are targeted at multiple devices, so devices must be differentiated first), multiple flows generated simultaneously are divided into different devices. Here, we take an isometric mapping approach to map source IP addresses in the case of both IPv4 and IPv6 in the samples so they can be divided into the same device. In addition, in the actual traffic environment, the ISP can synchronize the IPv4 address and IPv6 address of the same device based on the ADSL number or device number, so the interference caused by the dual-track IPs of a single device should not be considered.

However, from the perspective of third-party traffic classification, access is a kind of information that cannot be directly obtained in the actual network environment. We also have no way of predicting in advance the number of flows that each access contains or the total amount of access in the current time window.

In order to ensure QoE, data will be transmitted as soon as possible during application access, so the access life cycle is limited. Although a flow is a finite but unbounded time series, it can also be limited due to the finiteness of the data. In the actual network environment, the timeliness of ETC also makes it necessary to truncate a flow by using the sliding window on timestamps.

Therefore, in the process of flow clustering, we do not need to consider and should not consider clustering all the flows in a long period at one breath (or all the flows in the traffic file in the case of an offline classification scenario). The first step is to divide the flows by time window and cluster the flow in the time window. If a flow is divided into two time windows, feature merging or feature truncation can be used.

The multi-flow clustering process should not affect the subsequent classification process in the actual network environment. Hence, we use simple statistical features of the packet or flow granularity for flow clustering. Due to the immediacy of access, multiple flows in the flow bunch will also have a concentration in time. The correlation of data may also make the length and quantity of the packets correlated.

We selected 15 common statistical features. These statistical features include four categories: general protocol features, general time-dependent features, general quantity related features, and general length-dependent features, as shown in Table IV. In protocol features, we only use ports because the bunch may contain different transport layer protocols, but there may be similarities or increments between ports. Time-dependent protocols are our focus because flows of the same flow bunch are time-concentrated and similar in network environments. The quantity and length features are also theoretically suitable for flow clustering as they represent data, so we also included them in the feature significance experiment.

It is worth noting that our clustering goal is the current traffic, clustering it into multiple access. Even if some access

TABLE IV
ALTERNATIVE STATISTICAL FEATURES FOR FLOW CLUSTERING

Class	Feature name	Description (unit:flow)
General protocol	src_port	source port
features	dst_port	destination port
General time-dependent features	pkt_ts_first	timestamp of first packet
	pkt_ts_last	timestamp of last packet
	f_duration	flow duration
	max_iat	maximum IAT
	ran_iat	range of IATs
General quantity related features	std_iat	standard deviation of IATs
	send_pkt_ps	packet per second in sending
General length-dependent features	recv_pkt_ps	packet per second in receiving
	ran_pkt_len	range of IATs
	min_pkt_len	minimal packet length
	max_pkt_len	maximum packet length
	avg_pkt_len	average of packet lengths
	std_pkt_len	standard deviation of packet lengths

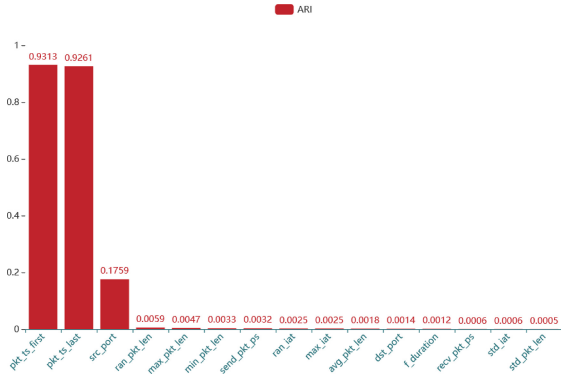


Fig. 3. Feature significance assessment chart with ARI.

belongs to the same application, these access units will be separated into different clusters. Furthermore, we evaluated the feature significance. We used standard hierarchical clustering to evaluate the clustering significance of single features. We used the average Adjusted Rand Index (ARI) as the evaluation criteria for features. The higher it is, the more influential the feature is.

Firstly, we used the basic hierarchical clustering method to randomly select parameters to evaluate the significance of the above features. Since the parameter gap was not considered in this process, distance_threshold was set as 2, the linkage was set as the ward, and the time window was set as 300 seconds. The graph of feature significance assessment is shown in Figure 3.

According to the evaluation results, the two features **pkt_ts_first** and **pkt_ts_last** are finally selected for clustering because they have the best feature significance. In essence, network traffic has the characteristics of sudden, so it is reasonable to use the timestamps of the flows for clustering. That is to say, multiple flows generated by a single access will be concentrated in a certain time interval, which is also the instant transmission of resources required by the application to ensure its QoE.

In the actual clustering process, we cannot predict the total amount of flows and access in the current time window. Therefore, distance-based clustering of fixed categories (such as K-means) should not be chosen. Therefore, the appropriate methods are density clustering and hierarchical clustering.

Algorithm 1 TSHC-SW

Input: sliding window size $size_w$, encrypted traffic to be clustered E , distance threshold parameter d

Output: bunches set $B = b_1, b_2, \dots, b_A$

```

1: foreach flow  $p$  in  $E$  then
2:    $pkt\_ts\_first\_list = get\_pkt\_ts\_first(p)$  //get the
   starting timestamp
3:    $pkt\_ts\_last\_list = get\_pkt\_ts\_last(p)$  //get the end-
   ing timestamp
4: end
5:  $E_{remain} = E$ 
6: while  $E_{remain} \neq null$  then
7:    $current\_ts\_first = pkt\_ts\_first\_list[0]$ 
8:   foreach flow  $p$  in  $E_{remain}$  then
9:     if  $Pkt\_ts\_first\_list[p] - current\_ts\_first \leq size_w$ 
       then
10:       $temp\_target += p$ 
11:       $temp\_features += [pkt\_ts\_first\_list[p],$ 
        $pkt\_ts\_last\_list[p]]$ 
12:    end
13:    else then
14:       $current\_ts\_first = Pkt\_ts\_first\_list[p]$ 
15:       $E_{remain} = E_{remain}[p:]$ 
16:       $windows += [temp\_target, temp\_features]$ 
17:    break
18:  end
19: end
20: end
21: foreach window  $w$  in  $windows$  then
22:   foreach flow  $p$  in  $w$  then
23:      $prox\_t = calculate\_proximity\_tensor(w)$  //cal-
       culate the correlation between each cluster
24:      $temp\_B += p$ 
25:   end
26:   while  $\exists i, j, i \neq j, distance(temp\_B[i], temp\_B[j]) < d$ 
       then
27:      $temp\_B = merge\_closest\_clusters(temp\_B)$ 
       //clusters merging through distances
28:   end
29:    $B += temp\_B$ 
30: end
31: return  $B$ 

```

Since encrypted traffic is a kind of structured non-Euclidean space discrete sequence data [39], we use a hierarchical clustering algorithm to cluster multiple flows. Therefore, we proposed a Time Sequential Hierarchical Clustering with Sliding Windows algorithm (TSHC-SW) based on agglomerative hierarchical clustering. The algorithm flow of the TSHC-SW is shown in Algorithm 1. The general idea of the algorithm is to use the similarity between various clusters to condense from dispersed massive clusters step by step gradually. Compared with the common agglomerative hierarchical clustering algorithm, it considers the burst characteristics of the traffic generated by each access, divides the traffic by time window, and carries on the continuous agglomeration with the timestamp as the feature. We did not use density clustering

because the time overlap and time interval distribution between flows is related to the network environment, and there will be noticeable differences in different applications, so it is not suitable to describe directly with simple density because this density is diverse.

The first step of the TSHC-SW algorithm is to form flows from the input packets. Then, the encrypted traffic is segmented according to the size of the sliding window, and each time window contains all the flows and truncated flows within the initial timestamp interval to obtain multiple windows (line 1 to line 20). Furthermore, within each time window, the correlation between each cluster is calculated, and if the distance between the two clusters is less than the distance threshold, they will be merged (line 21 to line 30). This procedure ends up with the output flow bunches. Density clustering also has the property that the clustering space is an arbitrary shape, so we use it as a control method to carry out experiments.

Note that a device may access multiple applications simultaneously, and there may be cross-traffic between different applications, especially between background applications and the currently used foreground applications. Therefore, there will inevitably be some tiny errors in the clustering process. However, in terms of time, the sudden occurrence of access will make the traffic very concentrated so that the flows can be effectively clustered. Multiple accesses to an application are clustered into different flow bunches.

B. Multi-Flow Application Classification Schemas by Flow Bunches

After obtaining the flow bunches by clustering, we need to classify each flow bunch further. However, it is undeniable that multi-flows are multiple single flows, and the single flow classification of encrypted traffic has already had many research works. Although the results of single flow classification are challenging to be used in the actual network environment, its classification effect has been satisfactory to a certain extent.

Therefore, schemas for bunch-oriented multi-flow application classification are proposed in detail. We summarize these schemas into the following five schemas.

In order to ensure the consistency of input while simultaneously utilizing the in-flow relationship and inter-flow relationship, the features in these five schemas are all formed by the combination of the packet length sequences of flows.

The first schema is to select a flow from the current flow bunch to directly construct the input feature for classification, which rules the classification process from multi-flow classification to single-flow classification and uses the classification result of the most representative flow to determine the classification result of the whole flow bunch. In this schema, we need to decide which flow to choose. Since the TSHC-SW model divides the time window, we can obtain the basic information of each flow in the current time window. The most straightforward idea is to choose the first flow because the first flow is the fastest to get. For many applications, especially in Web browsing, the first flow is the core flow. After obtaining the first flow, there is no need to store the features of subsequent

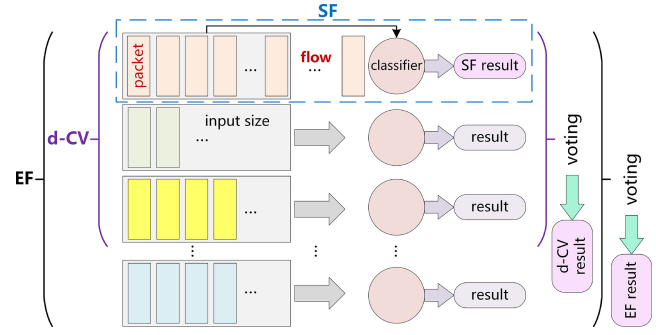


Fig. 4. Example graph of SF, EF, and d-CV.

flows further, and only need to maintain the flow bunch. In addition, it is feasible to select a flow randomly, but it is only operant when the packet length sequences of all flows in the flow bunch are cached. We compared the two choosing modes in a follow-up experiment. In theory, it is a pseudo multi-flow classification, but it can significantly improve the efficiency of pure single-flow classification. Furthermore, it contains the most fundamental advantages of multi-flow classification, making ETC results more usable than single-flow ones. This schema is referred to as the **Single Flow (SF) schema**.

The second schema is to classify each flow in the current flow bunch and carry out weighted voting on these single-flow classification results for multi-flow classification. This is a very time-consuming behavior, equivalent to reducing the multi-flow classification to multiple single-flow classifications. However, because of the flow bunch, we gain the ability to synthesize the results of each flow to get a real and valid result. In this case, because each flow needs to get the classification result, the classification process can be done in parallel with the clustering process. Therefore, this mode can theoretically achieve better classification effects and more usable classification results than simple single-flow classification. It is essentially a secondary classification of the results of multiple single-flow classifications. We call this schema **Each Flow (EF) schema**.

In the third schema, d candidate flows are selected from the current flow bunch for classification, and each is classified to acquire a result. Weighted voting is also performed on these d results. Similar to schema SF, this schema also needs to solve the problem of which flows to choose. The most naive option is the first- d flows, and the alternative is random- d flows. At the same time, the value of d also needs to be considered. This schema is between SF and EF, equivalent to balancing the efficiency of a single flow representing the whole flow bunch against the effect of all flows voting to get the result. A larger d requires more classification times and cache space but has a better effect. This schema is called **d -Candidate Voting (d-CV) schema**.

Noting that in the above three schemas, the size of the packet length sequence selected by each flow for classification is the same. Examples of these three schemas are shown in Figure 4.

The fourth schema is to classify the entire current flow bunch, which is relatively direct according to the clustering

results. In this schema, the packet length sequence of each flow should be selected to form a new feature for classification in theory, which is a pure multi-flow classification. It is because this schema directly utilizes the inter-flow relationship of multiple flows in the flow bunch, unlike EF and d -CV, which only indirectly utilize the classification results of corresponding flows. However, in the actual network environment, we need to face two problems. The first one is the consistency of the input size of the classifier. A trained classifier needs to have a fixed length input or encode it into a fixed input. In order to ensure the suitability of the classification model to different schemas, we do not consider the reencoding process in this paper. So we have to fix an input upper limit for the multi-flow classification. Another problem is that there is no way to know how many flows a flow bunch has or how many packets each flow has in advance. Hence, we have to fix the upper limit of the number of chosen flows in advance, similar to d in d -CV. At the same time, to ensure the fairness of flows (it is impossible to predict which flows in the flow bunch are more suitable for classification), we should combine the packet length sequence of the same size from each flow as far as possible.

In the above circumstances, we use each flow to provide a fixed-size packet length sequence and combine multiple sequences of equal size to form new features for classification. The new feature's total size is equal to that of a single flow in SF, EF, and d -CV (that is, the size of the packet length sequence). The fixed size provided by each flow is called each size. It is a variable parameter, but it needs to be divisible by the total size of the feature. This schema is called the **whole Flow (HF) schema**.

It is worth noting that the first four schemas are feature size independent, meaning there is no need to adjust the model's structural hyper-parameters (this is especially important in deep learning). Although retraining may be required under different schemas (which is not necessary, as demonstrated in subsequent experiments), the invariability of the model enables hardware and rapid dynamic schema adjustments, making the multi-flow classification more suitable for the actual network environment.

The fifth schema is an extended multi-flow classification schema, which combines d -CV with HF to introduce the inter-flow relationship without sacrificing part of the in-flow relationship. It directly selects d candidate flows from the current flow bunch and extracts each candidate's packet length sequence whose size is equal to the one in the SF schema for classification. The advantage of this schema is that it can utilize more features, but it multiplies the input, resulting in much overhead. Under this schema, because an inter-flow relationship should be considered, in addition to first- d and random- d , we add the practice of top- d . This choosing mode selects d flows with the largest packet counts, truncates it to feature size, and then combines and classifies it. This schema is called the **d -Candidate Together (d -CT) schema**.

Examples of HF and d -CT schemas are shown in Figure 5.

Theoretically, each of the five modes has its emphasis. The diagram of the five modes is shown in Figure 6. Therefore,

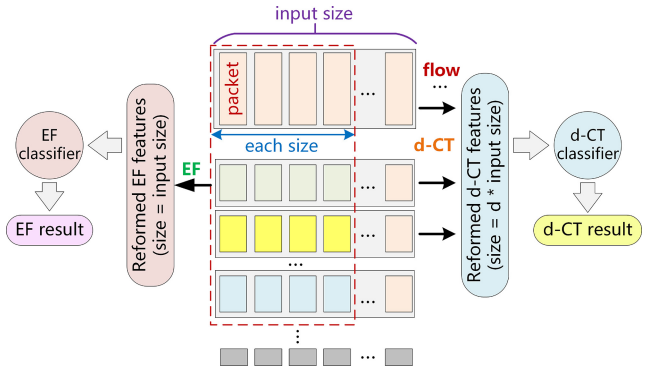


Fig. 5. Example graph of HF, and d -CT.

we conducted in-depth experiments to determine the effect of specific models.

It is worth noting that multi-flow classification involves multiple flows, each with a different number of packets. As a result, some flows may not meet the input requirements determined by one of the preceding schemas. Different schemas have different sample availability conditions. Even under some extreme parameters, all flows in the whole flow bunch cannot form the features required.

Existing studies tend to use data preprocessing to filter flows that do not meet the requirements, but in the open-world actual network environment, sample skipping may be caused. In order to effectively measure this situation, we proposed the concept of Sample Covering Rate (SCR). SCR refers to the proportion of bunch samples conforming to the input requirements in all samples to be classified under the current schema, which can be expressed as:

$$SCR = \frac{\text{Number of } \widetilde{S}_k}{M} \quad (2)$$

The higher the SCR, the better the adaptability to traffic in the actual network environment of the open world, the less sample neglect caused (which will directly cause the loss of classification information), and the better the global classification effect.

V. EXPERIMENTAL EVALUATION

In order to prove the effectiveness of multi-flow classification, we conducted experiments on flow clustering, single-flow classification baseline, and multi-flow classification.

A. Dataset and Experimental Settings

We conducted experiments on three datasets. First, we used the widely used ISCX VPN-nonVPN dataset [3] to prove the effectiveness of the classifier under single-flow classification.

The second dataset is the latest dataset based on access (with both access label and application category label) collected by us using the A3C system [37]. Based on this system, we can get the real label of every access and every flow of all applications directly on the end system at the same time. We named this dataset Cernet-Access-2022. This dataset was collected in the real network environment of CERNET, where

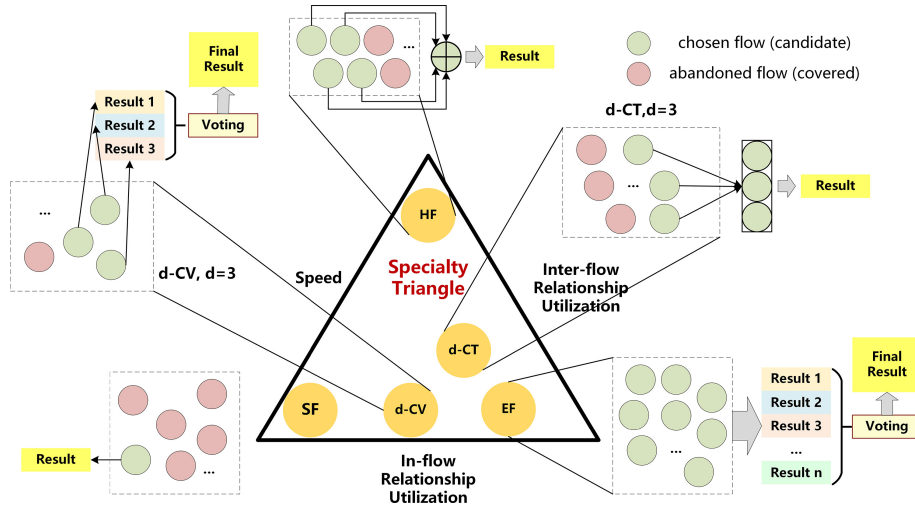


Fig. 6. Schematic diagram of five multi-flow classification schemas.

TABLE V
STATISTICS OF CORE CERNET-ACCESS-2022 DATASET (#: NUMBER OF)

Application category	#Access	#Flows	Average #flows per access	Service type
Sohu	3258	355377	109.08	Social media
Douban	1512	106386	70.36	Content forum
Bilibili	1200	40482	33.74	Online video
Sina	1455	140663	96.68	Portal website
NetEase	1133	52845	46.64	Online news
JD	639	44990	70.41	E-commerce
Hupu	1202	29105	24.21	Social platform
Youku	96	16597	172.89	Online video
Total	10496	786445	74.93	

user size and network infrastructure are similar to standard carrier networks. We collect pure traffic on real user devices, the collection paths including user manual access and script automatic access, and the devices involve three Windows PCs, two Linux workstations, and two Android mobile devices. The data is from May 2022 to November 2022 and covers the duration of the COVID-19 outbreak and the period after the outbreak has subsided. It is worth noting that, to ensure authenticity, we guarantee the time interval between each user manual access action real, while the automatic access interval of the script is smaller than the manual access interval to test the robustness of the clustering. The data from PCs and workstations in the dataset were all achieved by visiting the application's website with a browser, while the samples on mobile phones originated from native applications.

The details of the dataset are shown in Table V. Among them, since the unit of our multi-flow classification is one access of the application, we strictly counted the times of application access and the total number of flows to calculate the average number of flows per access.

On top of this, to further test the impact of clustering, we captured 700 times of access from 100 applications in June 2023 in the same environment, and the add-on is named CERNET-Access-2023-Aug. Due to the small sample size of each category, it does not conform to the training scenarios classified in this paper, so it is only used for testing clustering. The two datasets are published at <https://data.iptas.edu.cn/web/tbps>.

We chose a high-performance workstation as the verification platform for our experiments. It has an AMD 5950x CPU, 64GB memory, and a GTX 3090 GPU. It runs CUDA version 11.1, Pytorch version 1.8.1, and Python version 3.8.8.

In terms of evaluation criteria, classification and clustering have different evaluation criteria. In classification, we still use precision rate (Pr), recall rate (Rc), and F1-Score as the evaluation criteria.

Furthermore, we choose three machine learning (C4.5 [3], SVM [5], kNN), two ensemble learning (XGBoost, RF [7]), and four deep learning (CNN [8], Deep Packet [40], FS-Net [16], LS-LSTM [14]) methods as control methods to be used as classifiers in ETC for comparison.

In flow clustering, we selected a variety of common clustering evaluation criteria and combined them with the basic result statistics to describe the clustering effect in an all-round way. The main criteria include average purity, average Rand Index (RI), ARI, and average Adjusted Mutual Information Coefficient (AMIC).

In order to simulate the unknown and unbalanced situation of some applications in the open-world scenario, we selected six applications from eight categories as known data, namely our classification targets, and the other two categories as background traffic to interfere with the classification. To make the experiment more realistic, we choose Douban, Sohu, NetEase, Bilibili, JD, and Sina as the available data, while Youku (similar to Bilibili) and Hupu (similar to Douban) as the background interference traffic. It is important to note that we do not classify the other two types of traffic because, in this scenario, we do not know that they exist.

B. Experiments of Clustering Performance

Clustering is a process of unsupervised learning, so we do not divide the training set and test set from the sub-dataset but directly cluster the whole one. In TSHC-SW, since we need to define the size of the sliding window of the current traffic that needs to be clustered before the experiment starts, we first need to define an interval of the size of the sliding window. The significance of sliding windows is to ensure the clustering

TABLE VI
COMPARISON OF ALL CLUSTERING EFFECTS AMONG WINDOW SIZE VALUES IN FIXED DISTANCE THRESHOLD

Window size	Total cluster number	Avg purity	Avg flow number /window	Avg cluster number /window	Avg flow number /cluster number	Avg RI	Avg ARI	Avg AMIC
330	9026	0.98	756.3	12.13	62.34	0.99	0.95	0.97
360	8903	0.98	829.92	13.13	63.2	0.99	0.95	0.97
390	8799	0.98	888.92	13.9	63.95	0.99	0.95	0.97
420	8748	0.97	952.09	14.8	64.32	0.99	0.95	0.97
270	9309	0.98	623.82	10.32	60.45	0.99	0.94	0.96
300	9154	0.98	689.56	11.22	61.47	0.98	0.94	0.96
240	9634	0.98	556.56	9.53	58.41	0.98	0.93	0.96
210	10283	0.99	488.02	8.92	54.72	0.98	0.92	0.95
180	11937	0.99	431.51	9.15	47.14	0.97	0.88	0.92
150	13304	0.99	360.23	8.52	42.29	0.95	0.84	0.9
120	15780	0.99	291.55	8.18	35.66	0.93	0.78	0.86
90	20130	1	222.32	7.95	27.95	0.88	0.7	0.8
60	26820	1	151.83	7.24	20.98	0.76	0.54	0.65

effect with as few flows as possible. That is to improve the efficiency of clustering (prevent the appearance of busy waiting and reduce the required cache space). Hence, we set the lower limit of the sliding window to 60 (measured in seconds), the step to 30, and the maximum to 420.

In addition, since TSHC-SW is based on agglomerative clustering, the distance threshold should be set to limit the degree of agglomeration when the total number of clustering categories is unknown. Ten values of 4, 3.5, 3, 2.5, 2, 1, 0.5, 0.1, 0.05, and 0.01 are selected as alternatives. The optimal value of this parameter also needs to be verified by experiment.

Furthermore, for agglomerative clustering, we have to figure out the linkage type to measure the two most similar clusters. For encrypted traffic, it can be found from the dataset that there is not a significant difference among the number of flows of each access, so “ward” is selected as the linkage type, that is, to minimize the variance increased in the process of cluster merging.

We have conducted experiments on selecting the above parameters. First, under the condition of a fixed distance threshold (“1” was selected as the value of fixed distance threshold), experiments were conducted on different window sizes, and ARI was taken as the main evaluation criterion. The results of all evaluation criteria are shown in Table VI.

As can be seen from the results, bigger is only sometimes better. Although a small window will make it challenging to contain complete real flow bunches, thus bringing obvious systematic errors to clustering, a too large window will lead to the decline of clustering accuracy. The main reason is that when the amount of data to be clustered is larger, the relative distance between each cluster will become less “far” due to the dispersion of the overall distance, leading to the mis-clustering of the boundary flows.

Then, under the condition of fixed window size (300 seconds was selected as the size value), we conducted experiments on different distance thresholds. The experimental results under ARI as the evaluation criterion are shown in Figure 7.

On this basis, we comprehensively conducted experiments on all window sizes and distance thresholds and drew a 3D histogram to get the parameter selection under the best clustering effect. The clustering effect diagram under ARI as

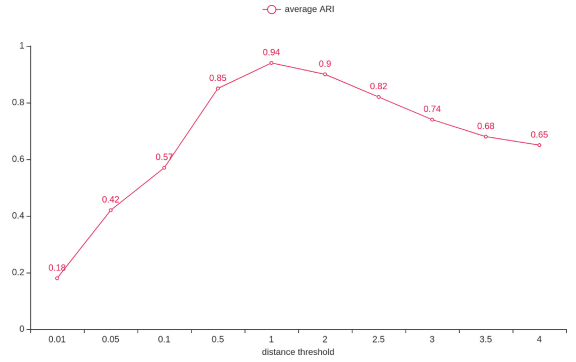


Fig. 7. Comparison of ARI among distance threshold values in fixed window size distance threshold with ward as the linkage type.

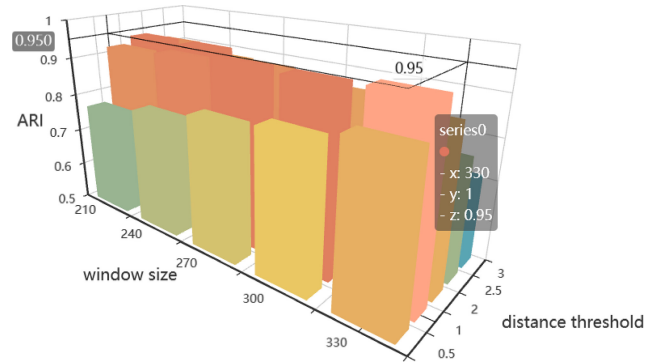


Fig. 8. 3D histogram of TSHC-SW clustering effect under ARI.

the evaluation criterion is shown in Figure 8, and the results under AMIC perform the same.

It can be seen from the results that the best one should be a window size of 330 and a distance threshold of 1. Therefore, we choose these two values as parameters of TSHC-SW.

Furthermore, to prove the superiority of TSHC-SW, we compare it with DBSCAN, a typical algorithm of density clustering methods commonly used in non-fixed category clustering. The optimal clustering results of the two are shown in Table VII (the optimal parameters of DBSCAN are also obtained by multi-parameter grid search).

It can be seen from the results that the TSHC-SW algorithm outperforms the DBSCAN algorithm and shows an excellent clustering effect in various indexes, which proves

TABLE VII
COMPARISON OF CLUSTERING EFFECT BETWEEN TSHC-SW AND DBSCAN METHOD

Methods	Window size	Parameters	Total cluster number	Avg purity	Avg flow number /cluster number	Avg RI	Avg ARI	Avg AMIC
TSHC-SW	330	n_clusters:None distance_threshold:1 linkage:ward	9026	0.98	62.34	0.99	0.95	0.97
DBSCAN	240	eps: 0.1 min_samples:5 algorithm:auto	10125	0.94	55.57	0.97	0.88	0.92

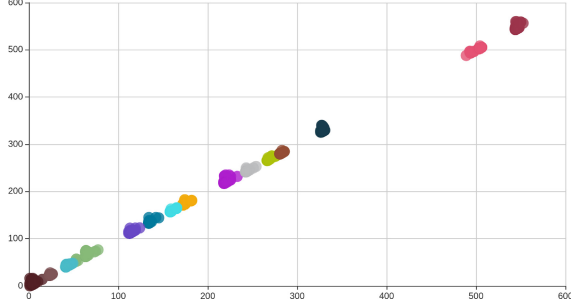


Fig. 9. Detailed clustering scatter distribution diagram under rectangular coordinate system of TSHC-SW (15 clusters).

that multi-flow classification is feasible and lays an essential foundation for multi-flow classification.

The clustering scatter distribution diagram formed by TSHC-SW is shown in Figure 9.

In an actual network, a single user may use a huge number of applications, which may run continuously in the operating system's background. Therefore, we further consider the influence of this situation on the clustering effect. Using the CERNET-Access-2023-Aug dataset, the clustering effect under different applications with limited continuous access is tested. The clustering ARI effect under different numbers of applications is shown in Figure 10. As can be seen from the figure, with the increase in the number of application classes, there is no apparent relationship between the number of application categories and the clustering effect, and ARI fluctuates periodically with the increase of categories. Within each fluctuation, the drop in ARI is caused by the inconsistency in the tightness of the timestamps of the flows for each application in the time window, which is related to the density of access, not the number of applications. Generally, the increase in application categories makes the clustering effect more significant. Hence, the applicability of our method in large-scale applications is guaranteed.

Since clustering is also time-consuming, we have conducted experiments on the time-cost performance of clustering. It is worth noting that since both clustering and classification require input reading of each flow, the clustering experiment only needs to consider the time cost of feature reading/writing and the core TSHC-SW algorithm. Experiments show that the throughput of clustering can reach 49,081 flows per second.

We use the clustering result as the input of the subsequent multi-flow classification.

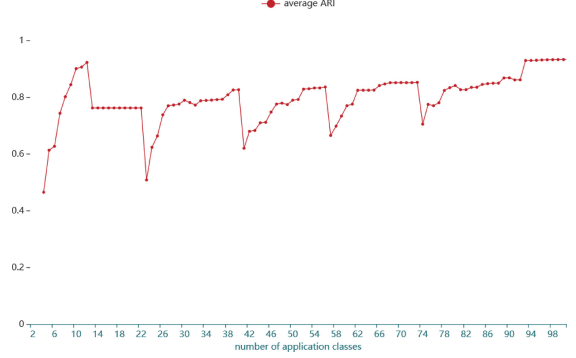


Fig. 10. Clustering ARI of TSHC-SW under different numbers of applications on CERNET-Access-2023-Aug.

C. Experiments of Classification Performance

Based on flow clustering, we need to select an appropriate classifier to classify encrypted traffic. To ensure fairness, we should use the same classifier for multi-flow and single-flow classifications. The input size should also be equal to ensure that the theoretical upper limit of information gain contained in the sample is consistent.

We test the effect of single-flow classification and take it as the baseline. It is important to note that this process is bunch independent, that is, classifying each flow purely by its application level label. It is worth noting that although deep learning classification methods are now mainstream, we still included traditional machine learning methods as the baseline. Because not all analysis nodes have GPUs in the Internet of Everything, in many cases, the actual network environment still uses machine learning methods as classifiers. In addition, under the Deep learning model, we did not use AppScanner, Deep Fingerprinting, and GraphDApp, because they had limited application scenarios and rigorous input requirements. Instead, we chose CNN, Deep Packet, FS-Net, and LS-LSTM, which are universal.

First, we experimented with machine-learning models. It is worth noting that training time and prediction time correspond to 80% sample quantity and 20% sample quantity, respectively, which is consistent with the training set and test set size. The bar chart for a comprehensive comparison of F1-score and training time is shown in Figure 11.

As can be seen from the results, C4.5, RF, and XGBoost have the best classification F1-score. Therefore, considering the classification effect and training cost comprehensively, we will prioritize using C4.5 and RF in the subsequent multi-flow classification.

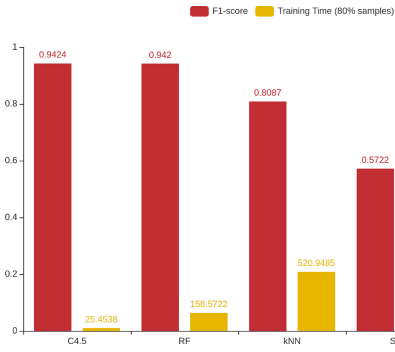


Fig. 11. Cross comparison of five machine learning and ensemble learning methods under F1-score and training time.

TABLE VIII
COMPARISON OF FOUR DEEP LEARNING METHODS
ON CERNET-ACCESS-2022 DATASET

Methods	Pr	Rc	F1	Training time (s)	Prediction time (s)
CNN	0.7894	0.7969	0.7824	52.0834	7.9734
Deep Packet	0.8158	0.8199	0.8117	58.2501	0.0630
FS-Net	0.9424	0.9426	0.9406	145.2141	0.6505
LS-LSTM	0.9473	0.9474	0.9461	49.9798	1.0838

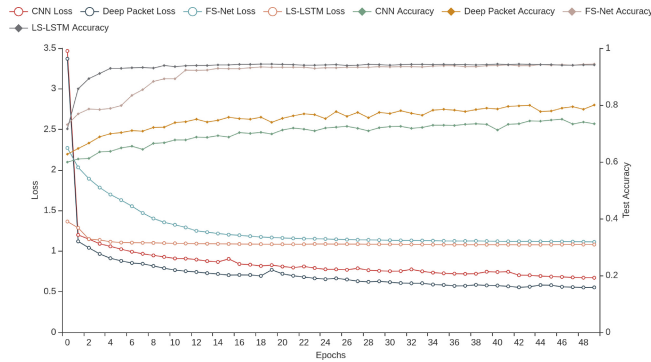


Fig. 12. Convergence line diagram of loss and test set accuracy of four deep learning models.

Further, we conducted experiments on the performance of four deep learning methods under the dataset. It is worth noting that we fully respect the limitations of input features and the selection of hyperparameters in previous studies. However, to ensure the fairness of input, we adjust CNN and Deep Packet slightly to adapt to the input of length sequence and keep the four models consistent in the initial input. The classification results are shown in Table VIII.

It is worth noting that epochs of CNN and Deep Packet are 200, while those of FS-Net and LS-LSTM are 50 due to the slow convergence of CNN and Deep Packet. It can be seen from the results that the two state-of-the-art deep learning methods (FS-Net and LS-LSTM) have better single-flow classification performance under the Cernet-Access-2022 dataset, and the training time is also within an acceptable range.

The convergence curve of classification results (including loss and test set accuracy) is shown in Figure 12. Results show that LS-LSTM not only has the best classification accuracy but also has the fastest convergence rate and the lowest initial loss, and FS-Net also has a faster convergence rate. The two methods are far better than CNN and Deep Packet



Fig. 13. Comprehensive comparison of four optimal methods on training time, prediction time and F1-score.

in classification accuracy and convergence speed. Therefore, FS-Net and LS-LSTM are more suitable for ETC and are preferred in multi-flow classification.

However, compared with FS-Net, LS-LSTM's advantage in terms of time consumption and classification effect is not free lunch. It has a large model size (98.67 MB), while FS-Net only has less than one-third of that (29.32 MB). Therefore, we choose two models, FS-Net and LS-LSTM, as participants of deep learning for subsequent multi-flow classification.

Then we compare the above optimal machine learning method, optimal ensemble learning method, and state-of-the-art deep learning method. The results in F1-score, training time and prediction time cost are shown in Figure 13.

The above results will be used as a baseline for multi-flow classification.

D. Performance Evaluation of Different Bunch Classification Schemas

Based on determining the classifier of ETC, we finally need to evaluate the five different bunch classification schemas in multi-flow classification.

Because there are certain differences between the inputs of multi-flow and single-flow classification, which are both reflected in the training and prediction stages, we further summarized five different flow bunch classification schemas into three training modes in the training stage. They are single-flow training (corresponding to SF, EF, and d -CV in prediction), multi-flow training (corresponding to HF), and d -candidate multi-flow training (corresponding to d -CT).

It is worth noting that in the flow bunch, we want all flows to belong to the same access as much as possible, so we use the flow bunch obtained after flow clustering as the classification unit. However, because there are some errors between the flow bunch and the real access flows, it is necessary to consider the influence of such errors on classification results.

Purity directly reflects the number proportion of data belonging to the cluster in each cluster. **Therefore, we need to multiply purity as a constraint factor to reflect the negative effect of the flow clustering process on multi-flow classification. According to the clustering experiment in the first part, the value of this constraint factor is 0.98.**

First of all, we experiment with three multi-flow classification schemas under single-flow training.

TABLE IX
COMPARISON OF SINGLE-FLOW TRAINING WITH SF PREDICTION
SCHEMA (FIRST-FLOW AND RANDOM-FLOW) TO BASELINE
IN FOUR METHODS

Choosing mode	Methods	F1	Prediction time (s)
First-flow	C4.5	0.9323	0.1173
	RF	0.9440	0.1973
	FS-Net	0.9141	0.1187
	LS-LSTM	0.9367	0.1977
Random-flow	C4.5	0.9336	0.1275
	RF	0.9234	0.2182
	FS-Net	0.8949	0.1189
	LS-LSTM	0.9197	0.1992

TABLE X
COMPARISON OF SINGLE-FLOW TRAINING WITH EF PREDICTION
SCHEMA TO BASELINE IN FOUR METHODS

Method	Pr	Rc	F1	Prediction time (s)
C4.5	0.9431	0.9415	0.9408	0.7342
RF	0.9456	0.9442	0.9437	66.5083
FS-Net	0.9161	0.9133	0.9106	96.7742
LS-LSTM	0.9323	0.9300	0.9287	298.5143

1) *SF Schema*: The first-flow and random-flow methods are adopted in the SF schema to select the flow bunch representation. The classification results are shown in Table IX.

It is worth noting that, in order to level out the target number of predictions and make the coverage of flows classified by the two flow-choosing modes under baseline and SF consistent, we adjusted the prediction time in the baseline results and SF results to make the total number of flows the same, which includes 43,853 flows and 9,026 bunches. For prediction, it is 20% samples (1806 bunches). Flows that do not meet input requirements are screened. Therefore, the number of flows participating in the classification is far less than the total number (740,743 flows).

It can be seen from the results that compared with the baseline, the SF schema does not bring significant improvement in classification effect and even reduces some, especially in deep learning methods. However, it significantly improves prediction efficiency. When using the SF schema, the prediction time is reduced to more than 10% compared to the baseline. This prediction time saving is even more significant if we include the filtering of flows that do not meet input requirements. However, the random-flow choosing mode consumes more prediction time than the first-flow because it needs to preselect one flow with the random machine. In addition, the effect is worse than that of the first mode. The reason for this phenomenon may be that some randomly selected flows have insufficient feature significance, and their feature boundary is in a weak position compared with the first flow in the training process, resulting in a decline in classification F1-score.

Therefore, it is important to take first-flow choosing mode when using SF schema. In addition, first-flow is more in line with online classification requirements. As long as the packet sequence of the first flow that meets the input requirements is taken as the input, there is no need to wait for other flows, which significantly saves the cache space.

2) *EF Schema*: We also aligned the prediction time, and the classification results are shown in Table X.

As can be seen from the results, EF schema improves classification accuracy and recall rate, and there is a nice improvement of more than 2% if we only think about classifications. However, the improvement is limited after considering the negative effect of clustering. Nevertheless, the prediction time has been dramatically raised, especially in RF, a relatively complex ensemble learning method. The prediction time has reached 300 times the baseline, which is unacceptable.

Moreover, the classification performance in deep learning decreases compared to the baseline approach, mainly because the deep learning's dropout operation to ensure generalization capability reduces the classification accuracy of low-significance flows to some extent. However, the result of low significance flow has the same weight as that of high significance flow in the final voting, which leads to the wrong final classification result of the whole flow bunch, resulting in the decline of the overall classification effect. Moreover, the complexity of the deep learning model itself leads to a catastrophic increase in prediction time.

Therefore, the application of EF schema is very narrow in the actual network environment. It is suitable only when the number of flow bunches and samples to be classified is tiny, and the computing resources are sufficient to produce. This schema is more suitable for traditional machine learning methods.

3) *d-CV Schema*: *d-CV* schema also contains the first and random schemas and different *d* values ranging from 2 to 5. The reason why 5 is the maximum value is that category disappearance will occur if we go higher. In other words, samples of a certain category do not meet the input requirements and are discarded, indicating the model's unavailability. The prediction results under *d-CV* are shown in Table XI.

It can be seen from the results that, on the whole, the classification effect is better than that of the baseline except for the FS-Net in *d* = 2. The results indicate that the *d-CV* schema can effectively improve classification performance. However, the cost is exceptionally high in prediction time. From the schema perspective, the main reason is that it has two more processes. The first is selecting *d* candidates, and the second is the multiple classification times and subsequent voting compared with SF.

Consistent with SF, compared with the first mode, the random mode has less classification effect. It also slightly increases the time consumed by prediction, proving that flows generated first will have a higher significance in a single access.

Furthermore, we compare three different schemas under single-flow training horizontally.

First, we horizontally compare the F1-score under three schemas in single-flow training, picking the first choosing mode for comparison, as shown in Figure 14.

It can be seen from the results that the *d-CV* schema has the best classification effect, and with the improvement of *d*, the classification F1-score continues to improve. However, the proportion of improvement has been decreasing. As *d* gets larger and larger, the improvement of the classification effect will be more and more limited. In the extreme case of EF, the classification effect is worse than all the *d-CV* results.

TABLE XI
COMPARISON OF SINGLE-FLOW TRAINING WITH d -CV PREDICTION
SCHEMA TO BASELINE IN FOUR METHODS

Choosing mode	Candidate count	Methods	F1	Prediction time (s)
First	$d = 2$	C4.5	0.9535	0.5997
		RF	0.9535	56.7869
		FS-Net	0.9289	83.6961
		LS-LSTM	0.9462	256.9762
	$d = 3$	C4.5	0.9685	0.5011
		RF	0.9678	47.2852
		FS-Net	0.9624	63.6463
		LS-LSTM	0.9675	216.4696
	$d = 4$	C4.5	0.9720	0.4416
		RF	0.9739	40.6813
		FS-Net	0.9609	59.4059
		LS-LSTM	0.9721	182.3561
	$d = 5$	C4.5	0.9724	0.3619
		RF	0.9749	32.8773
		FS-Net	0.9650	47.4929
		LS-LSTM	0.9713	145.4624
Random	$d = 2$	C4.5	0.9389	0.6147
		RF	0.9493	59.8586
		FS-Net	0.9048	81.6458
		LS-LSTM	0.9287	251.3400
	$d = 3$	C4.5	0.9634	0.566454
		RF	0.9659	52.745923
		FS-Net	0.9533	64.804786
		LS-LSTM	0.9604	216.030875
	$d = 4$	C4.5	0.9616	0.4768
		RF	0.9684	42.9590
		FS-Net	0.9452	57.7812
		LS-LSTM	0.9614	176.9440
	$d = 5$	C4.5	0.9686	0.3660
		RF	0.9725	32.3607
		FS-Net	0.9610	47.0764
		LS-LSTM	0.9700	144.6205

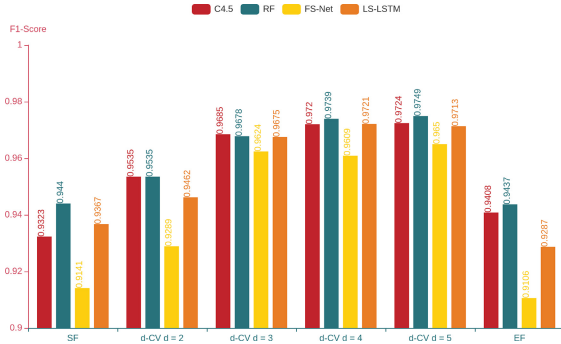


Fig. 14. Comparison of three schemas under single-flow training of four models on F1-score.

In addition to the reasons mentioned above for low significance flow, another reason is that EF does not limit the number of flows each flow bunch can acquire, which also causes the number of voting results used by different flow bunch to be inconsistent. Many participants will vote if a bunch is large and many flows satisfy the input criteria. This situation will further magnify the influence of low significance flows in the case of a large flow bunch because, in d -CV, only d candidate flows are selected even for a large flow bunch.

Then we compare the prediction times of d -CV and EF horizontally, as shown in Figure 15.

The results show that EF has the highest predictive time, which is expected because it classifies all the flows at once. As d goes up, theoretically, the prediction time should go up,

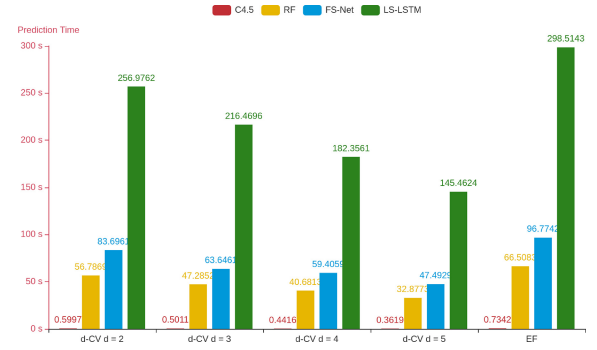


Fig. 15. Prediction time comparison of d -CV and EF schemas.

but the actual prediction time goes down. This phenomenon is because although the number of candidates selected from each bunch is increasing exponentially, the number of flow bunches that can participate in the classification is decreasing. Some of the flow bunches that fail to select d flows to meet the input conditions are skipped. This situation will be further elaborated in the subsequent experiments on SCR.

4) *Multi-Flow Training and HF Schema*: There are obvious differences between multi-flow training and single-flow training. It lies in that multi-flow training considers multiple flows and pursues the same feature space size as single-flow training. In other words, multi-flow and single-flow training have the same requirements for the packet length sequence input size. Hence, we need to get the packet length sequence from not one but multiple flows after determining which flow needs to be chosen.

In the training process, since we cannot predict the number of packets of each flow in advance, we can only limit the number of packets of each flow to the subsequent classification, which is called each size θ (this is consistent with the result of selecting k flows with fix value k as the meta-input of multi-flow classification). To ensure a uniform input size and meet the fixed input requirements of the neural network, we better make sure that each size should be divisible by the input size so that the number of packets provided by each flow is fair.

In the actual network environment, we cannot predict the total number of flows in advance, so selecting flows randomly and extracting packets according to each size is not appropriate. Therefore, we consider using adjacent flows for sample formation, and this approach is particularly beneficial in the case of open-world online classification.

In multi-flow classification, FS-Net would suffer model crashes, so we have modified it to forgo reconstruction loss.

We selected $\theta = 10$ and $\theta = 25$ as options of each size to conduct training and testing in the multi-flow training scenario. The results compared with the single-flow are shown in Table XII. It is worth noting that the training data of multi-flow classification is not subject to the purity value reduction because it is trained according to the exact flow bunch, not the flow bunch obtained by clustering.

The results show that, on the whole, compared with the single-flow, the effect of multi-flow classification is better, mainly reflected in the precision rate, especially in ensemble learning. When $\theta = 10$, RF's classification effect is greatly

TABLE XII
COMPARISON OF MULTI-FLOW TRAINING WITH SINGLE-FLOW
TRAINING BASELINE WITH DIFFERENT θ

θ	Sample Size	Methods	F1	Training time (s)
10	235086	C4.5	0.9675	342.6179
		RF	0.9972	2587.7309
		FS-Net	0.9991	775.2768
		LS-LSTM	0.9911	231.9407
25	130587	C4.5	0.9408	153.9406
		RF	0.9437	980.3356
		FS-Net	0.9845	429.2449
		LS-LSTM	0.9752	132.3215

improved. However, due to the loosening of its flow input restriction, the number of available samples is greatly increased, and the consumption of I/O is also grown so that the training time is significantly raised. The maximum increase multiple can be up to 17 times, which is about proportional to the square of the number of samples. Since multi-flow and single-flow classification has different input restrictions for prediction, the prediction time will be compared in subsequent experiments.

From the perspective of classification accuracy, both $\theta = 10$ and $\theta = 25$ can improve the effect, especially when $\theta = 10$. However, the cost is that in the same sample space with the unit of flow, the number of samples in the multi-flow training mode is much larger than that in the single-flow training mode (even if the samples are constructed in the way of a sliding window, not random). The main reason is that classification under multi-flow training can give more consideration to the relationship between flows in a bunch than single-flow classification, even if the packet input of each flow is reduced, sacrificing the expression of a part of the Markov relationship in the flow. The gains make up for the lack of it, especially when θ is small.

$\theta = 10$ has a better classification effect than $\theta = 25$. However, it is worth noting that the TLS-1.2 handshake contains seven packets, and augmented to the TCP three-way handshake, it all contains ten packets. Therefore, the minimum value of θ should be at least 10. The above phenomenon is consistent with the assumption that multiple flows generated at one access better represent the access than a single flow, even with the same feature space.

Furthermore, we experimented with multi-flow classification in HF schema, including $\theta = 10$ and $\theta = 25$. There is not much difference in the number of samples with different categories, each of which selects an input from each flow bunch. It is worth noting that there will be sequence fragment repetition under certain circumstances (which has a certain probability of improving the classification effect very limited, depending on the characteristics of the model). Regardless, the whole sample will only be partially alike to avoid the result invalidation caused by the consistency of the training set and test set. Detailed classification results based on categories of $\theta = 25$ are shown in Table XIII, which includes the number of predicted samples.

As can be seen from the results, under the condition of $\theta = 25$, the classification result under multi-flow training is outstanding. No matter what kind of application, they all have good classification accuracy, which is the same in each model.

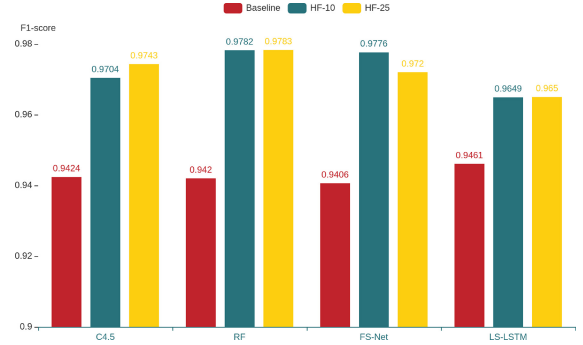


Fig. 16. Longitudinal comparison of four models on F1-score under HF schema with baseline, $\theta = 10$ and $\theta = 25$.

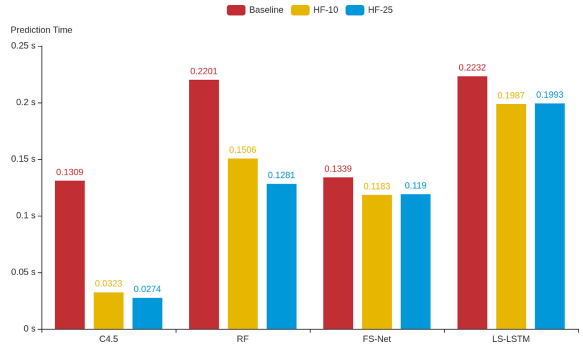


Fig. 17. Longitudinal comparison of four models on prediction time under HF schema with baseline, $\theta = 10$ and $\theta = 25$.

This result reflects the superiority of multi-flow training over single-flow training. With the same input size, the introduction of features of inter-flow relationships greatly improves the classification effect. It is worth noting that this is the result without purity reduction because it can only be applied to the whole and cannot be carried out for a specific application. Purity can only indicate the fit degree between the sample of the whole cluster and the real category but cannot represent the degree of aggregation of the sample of a specific application category and its class.

Next, we conducted a longitudinal comparison of different models and θ in the HF schema of multi-flow training. The first is F1-score, as shown in Figure 16.

It can be seen from the results that HF schema significantly improves the classification effect, which is mainly due to the feature superiority brought by multi-flow training. It directly proves that multi-flow classification outperforms single-flow classification.

We then compare the prediction time in the HF multi-flow classification model with the baseline time, as shown in Figure 17.

The results show that, under the same input size, the prediction speed of multi-flow classification is faster than that of single-flow classification, which indicates that multi-flow classification, especially HF schema multi-flow classification, is more satisfactory than traditional single-flow classification in terms of prediction time and classification effect.

5) *d-Candidate Flow Training and d-CT Schema*: Finally, we experimented with *d*-candidate flow training, another multi-flow classification training mode. Similar to multi-flow

TABLE XIII
DETAILED APPLICATION-LEVEL COMPARISON OF FOUR MODELS ON PRECISION, RECALL, AND F1-SCORE UNDER HF SCHEMA WITH $\theta = 25$

Application	Sample Size	C4.5			RF			FS-Net			LS-LSTM		
		Pr	Rc	F1	Pr	Rc	F1	Pr	Rc	F1	Pr	Rc	F1
Douban	293	0.9966	0.9932	0.9949	1.0000	0.9966	0.9983	1.0000	0.9964	0.9982	0.9802	0.9834	0.9818
Sohu	652	0.9939	0.9969	0.9954	0.9985	0.9985	0.9985	0.9868	0.9967	0.9917	0.9876	0.9969	0.9922
Netease	156	0.9936	0.9936	0.9936	1.0000	1.0000	1.0000	0.9868	1.0000	0.9934	0.9752	0.9874	0.9812
Bilibili	263	0.9962	0.9962	0.9962	1.0000	1.0000	1.0000	0.9950	1.0000	0.9975	0.9906	0.9769	0.9837
JD	95	0.9840	1.0000	0.9895	0.9947	0.9960	0.9970	1.0000	1.0000	1.0000	0.9903	0.9714	0.9808
Sina	257	0.9883	0.9883	0.9883	1.0000	0.9895	0.9947	0.9923	0.9627	0.9773	0.9820	0.9681	0.9750
weighted avg (unreduced)		0.9942	0.9942	0.9942	0.9923	1.0000	0.9961	0.9919	0.9919	0.9918	0.9848	0.9847	0.9847

TABLE XIV
COMPARISON OF d -CANDIDATE FLOW TRAINING WITH SINGLE-FLOW TRAINING BASELINE WITH DIFFERENT d

Methods	d	Sample Size	F1	Training time (s)
C4.5	2	35760	0.9495	51.2364
	3	28893	0.9610	69.4468
	4	22997	0.9616	83.9776
	5	18030	0.9660	96.8626
RF	2	35760	0.9649	200.2491
	3	28893	0.9699	234.1801
	4	22997	0.9735	233.0365
	5	18030	0.9786	228.1679
FS-Net	2	35760	0.9447	156.8874
	3	28893	0.9873	160.3979
	4	22997	0.9249	153.5513
	5	18030	0.9529	228.1679
LS-LSTM	2	35760	0.9725	57.0040
	3	28893	0.9778	61.5383
	4	22997	0.9919	59.5242
	5	18030	0.9883	57.6058

training, it also obtains the flow to be trained through a sliding window. The comparison of training results with traditional single-flow training baselines under different values of d is shown in Table XIV.

It can be seen from the results that d -candidate flow training, as another form of multi-flow classification, has far better test results than single-flow classification. The advantages of multi-flow classification are further explained, and this process does not sacrifice part of an in-flow relationship like multi-flow training. It can be shown that the inter-flow relationship is effective for classification. However, the increase in training time is also not negligible. Normally, with the increase of d , the classification result of the test set should get better, but there are some exceptional cases. For FS-Net, when $d = 4$ and 5, the number of samples of some categories is too small, which leads to the collapse of the model, and thus the overall classification accuracy decreases seriously.

Furthermore, taking the LS-LSTM model as an example, we carefully compared the relationship between sample size, training time, and test set F1-score, as shown in Figure 18. It should be noted that the purity reduction was not carried out for F1-score because it was an intra-model comparison.

It can be seen from the results that the larger d is, the fewer available samples are, and the training time will first increase and then decrease. It is because SCR and feature space size weigh each other. The slight decline of LS-LSTM classification F1-score is also due to the overfitting of other categories caused by too few samples when $d = 5$.

The schema corresponding to d -candidate training is d -CT. Although there are differences in training modes, d -CT and d -CV are similar, both facing the situation that possible

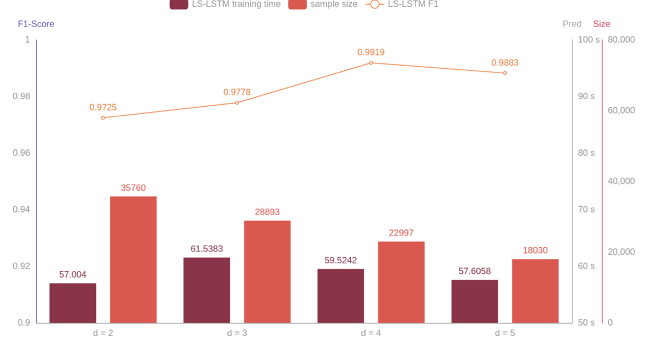


Fig. 18. Comparison of sample size, training time, and test set F1-score of LS-LSTM under d -CT schema with different d .

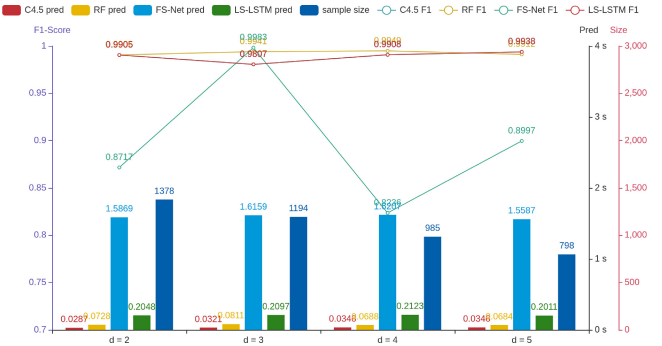


Fig. 19. Comparison of F1-score, sample size, and prediction time under d -CT schema along with first choosing mode with different d .

samples in the flow bunch are not enough to support classification.

We first experimented with the first choosing mode under the d -CT schema. The comparison of the F1-score, model size, and prediction time of four models under four different values of d is shown in Figure 19. The purity reduction of the F1-score was also not carried out.

It can be seen from the results that the number of available samples decreases significantly with the increase of d . The category F1-score showed slight improvement (FS-Net's F1-score fluctuation was due to model collapse, consistent with d -candidate training). Excessive in-flow relationships will not bring significant improvement to classification because they are sufficient for classification, and the improvement mainly comes from inter-flow relationships. The overall prediction time remains mostly the same under different d , which is also consistent with d -candidate training. Therefore, in the process of using d -CT schema, the impact caused by the decrease in available samples cannot be ignored.

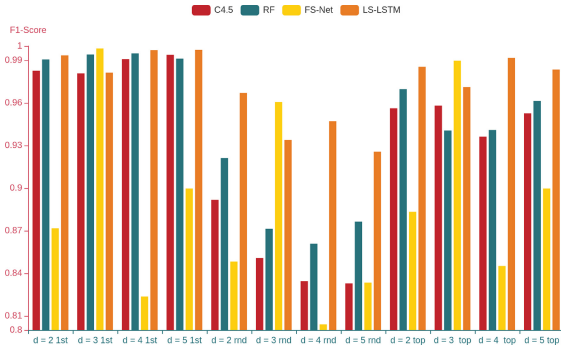


Fig. 20. Comparison of F1-score under d -CT schema among first, random, and top choosing mode with different d .

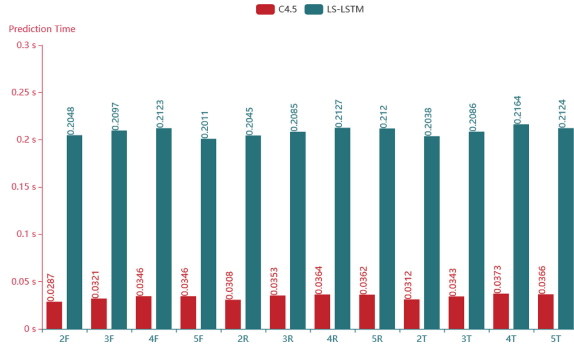


Fig. 21. Comparison of prediction time under d -CT schema among first, random, and top choosing mode with different d .

Furthermore, we compare the relationship between three different flow selection modes under d -CT, as shown in Figure 20.

The results show that the random choosing mode is still the worst, while the first mode has the best classification effect on the whole, but for FS-Net, the top mode is better than the first mode in all d values except $d = 3$. It indicates that the top mode introduced in d -CT will have better performance under partial models, indicating the local applicability of the top mode.

Then the prediction time of three different flow selection modes is given, as shown in Figure 21, where 2F refers to the first choosing mode for $d = 2$ (R and T stand for random and top, respectively).

The difference in prediction time is mainly reflected in the model. At the same time, different choosing modes have little influence in the case of large data volume classification, such as d -CT. The main influencing factor lies in the number of available samples and the size of d , that is, the feature input size.

In classification effect, d -CT and HF are the best of the five schemas. These two schemas belong to different multi-flow training modes, reflecting the superiority of multi-flow classification. However, in the actual deployment process, d or θ should be determined according to the actual situation.

In addition to the classification effect, we also conducted experiments on the model size, a system resource parameter closely related to the open-world actual network environment. The model size of four different models in three different training modes is shown in Figure 22.

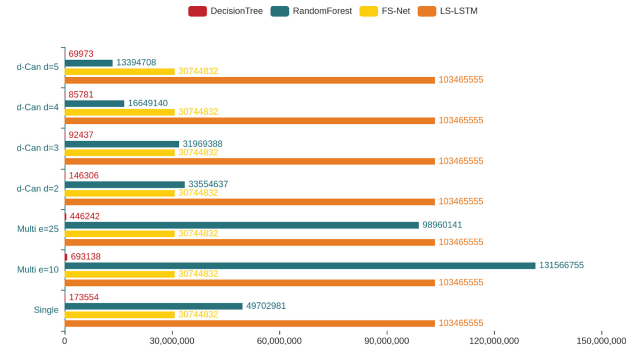


Fig. 22. Comprehensive comparison of model size of four models on three training modes.

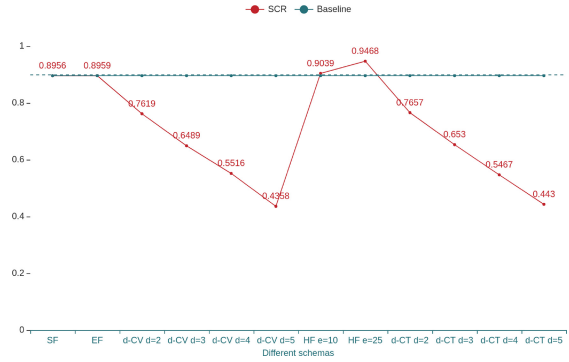


Fig. 23. Comprehensive comparison of SCR under five multi-flow schemas with all chosen d and θ .

It can be seen from the results that, among machine learning and ensemble learning, the multi-flow training model is the largest, mainly because there are more samples for training. The size of the d -candidate flow training model decreases with the increase of d , because fewer and fewer samples are available. For the deep learning model, due to the fixed structure of the neural network, the model size is consistent (sample independent). In multi-flow training, RF models are larger than the huge LS-LSTM models when $\theta = 10$. Therefore, the deep learning method will have better applicability in an open-world environment with massive data and multi-flow classification.

Then, we comprehensively evaluated SCR under different schemas, including SCR baselines for single flow classifications, as shown in Figure 23.

The results show that the EF and SF's SCRs are comparable to the SCR baseline of the single flow classification because they are single flow classifications with the extension of flow bunch. The SCRs of d -CV and d -CT were similar, and the higher the d , the lower the SCR because there were not enough samples in the flow bunch. HF's SCR is higher than the baseline because the number of packets required per flow has decreased. The reason why $\theta = 25$ is higher than $\theta = 10$ is that if θ is obtained too small, it will lead to too many flows required for each flow bunch and fall into the same problem of insufficient flows faced by d -CV and d -CT. Hence, the HF schema is the best from an SCR point of view alone.

SCR is a necessary but currently neglected metric in the open-world actual network environment. The lower the SCR, the more samples will be ignored in the actual

classification, resulting in incomplete classification results. It reflects the superiority of multi-flow classification over single-flow classification from another aspect.

E. Discussion and Analysis

1) *Why Are EF and d-CV Schemas' Prediction Times So Many Times Longer Than the Baseline, Which Should Not Be in Theory?:* Although theoretically, the prediction time's increase is due to the growth in the number of classifications and the voting process. However, another external factor that dramatically increases the prediction time in implementing the method is the repeated loading of the model. When classifying with flow bunch as the unit, the model needs to be loaded before each voting to ensure the purity of the result to be voted. This action is widespread in the open-world actual network environment. However, the additional overhead of this implementation can be greatly reduced if caching and parallel computation are used, that is, to compute and split the bunch simultaneously (but may result in computation wastage in d-CV schema). Therefore, this results in a dramatic increase in prediction time for RF and the two deep learning methods, which is not evident in C4.5 due to its small model size.

2) *Why Is the First Choosing Mode More Effective Than the Random Choosing Mode, But the Difference Between the Two Is Insignificant in the d-Candidate, Especially When d Is Large?:* This phenomenon is due to the implementation pattern of applications, especially Web pages. HTML and core resources are prioritized to ensure the user's QoE. In this way, even if the network is not good, it can still enable users to access it as early as possible. Therefore, the previous flows in each access are, in most cases, more significant because they are tied to the application's core data. The subsequent flow is likely to be irrelevant or duplicate with other applications' aesthetic data, external link data, and even advertising. However, in the two schemas of d-candidate, the larger d is, the more flows need to be selected in each flow bunch. When the total number of flows in the flow bunch is certain, the random choosing mode will gradually move closer to the first choosing mode because the probability of selecting the front flows increases with the selection times. As a result, the larger d is, the smaller the gap between them.

3) *Since Simple Multi-Flow Classification (Training) Can Get Good Results, Is Flow Clustering Worthwhile?:* Undeniably, the test set classification results in multi-flow training are already very good, far better than single-flow classification. However, it is worth noting that the premise is that we already have real access-level labels and application labels. Hence, the selection of flows in multi-flow training is completely correct, and there will be no case of choosing flows belonging to two different applications to classify. However, in the actual network environment, it is impossible to know the distribution of flow bunch without flow clustering. So clustering is not only worthwhile but also the core basis of multi-flow classification. On the other hand, flow clustering can also make the classification results more available than single flow classification, and it can also make it possible to

dynamically adjust the demand (pursuing classification speed or classification accuracy).

However, the flow clustering process comes at a cost. Its throughput is approximately 49,081 flows per second, while the LS-LSTM in the fastest HF schema has a throughput of approximately 46,501. In other words, if a deep learning model is deployed, although the throughput of flow clustering is higher than that of the classification model, the time expanse cannot be ignored if serial processing is used. However, in the actual network environment, the deep learning classification model relies on the computing power of the GPU, while the clustering relies on the CPU, and the two can even be processed in parallel on the same device. If multiple devices are used, the two can be fully parallelized, thus removing the time overhead effect of clustering on classification.

4) *In the Actual Network Environment, How to Choose a Schema and Balance the Impact of Low SCR?:* It can be seen from the experimental results that HF has the best classification effect, the fastest classification speed, and the highest SCR. However, it has a potential premise: it needs to ensure that each flow bunch has enough flow. This premise can be guaranteed under normal circumstances, but the effect of HF may not be guaranteed under special situations like sampling or asymmetric traffic scenarios. Moreover, HF must conduct multi-flow training in advance, which must also be considered. Although d-CT has a good effect, its low SCR makes it more suitable for the scenario of heavy flow classification. It focuses on the application with a long flow, such as the application under the online video service. For SF, EF, and d-CV, they can meet different needs through the external clustering model without changing the original classification model. It can be a balance between the classification effect, speed, and model size, which can be selected according to the actual demand.

VI. CONCLUSION

In this paper, we first carefully excavate the drawbacks of the existing single-flow ETC methods. To solve this problem, we first analyze the application data transmission pattern of encrypted traffic and determine the multi-flow classification idea based on access. Furthermore, from the perspective of information entropy and information disclosure, we clarified the representation of in-flow and inter-flow relationships under the length sequence features, providing a theoretical basis for multi-flow classification. Furthermore, since the flow bunch can not be obtained directly, we propose the TSHC-SW algorithm to obtain the flow bunch. In the end, we proposed five different multi-flow classification schemas. Each schema has its specialties, and the pure multi-flow classification schema HF is superior to the traditional single-flow classification in classification effect, efficiency, and SCR. Experimental results show that the proposed flow clustering and multi-flow classification methods outperform the existing state-of-the-art methods. It opens up a new research field for ETC.

The current multi-flow classification is not perfect, and there are some special schemas that we do not consider at present. At the same time, we did not introduce the graph feature, an emerging aspect of encrypted traffic feature engineering. In the

future, we will continue researching multi-flow classification and flow clustering to achieve an open-world ETC with better results and more adaptive ability.

REFERENCES

- [1] Google. "HTTPS encryption on the Web—Google transparency report." 2022. [Online]. Available: <https://transparencyreport.google.com/https/overview>
- [2] B. Yang and D. Liu, "Research on network traffic identification based on machine learning and deep packet inspection," in *Proc. IEEE 3rd Inf. Technol. Netw. Electron. Autom. Control Conf. (ITNEC)*, 2019, pp. 1887–1891.
- [3] G. Draper-Gil, A. H. Lashkari, M. S. Islam Mamun, and A. A. Ghorbani, "Characterization of encrypted and VPN traffic using time-related," in *Proc. 2nd Int. Conf. Inf. Syst. Security Privacy (ICISSP)*, 2016, pp. 407–414.
- [4] J. Yang, J. Narantuya, and H. Lim, "Bayesian neural network based encrypted traffic classification using initial handshake packets," in *Proc. 49th Annu. IEEE/IFIP Int. Conf. Depend. Syst. Netw. Supplement. Vol. (DSN-S)*, 2019, pp. 19–20.
- [5] J. Muehlstein et al., "Analyzing HTTPS encrypted traffic to identify user's operating system, browser and application," in *Proc. 14th IEEE Annu. Consum. Commun. Netw. Conf. (CCNC)*, 2017, pp. 1–6.
- [6] W. Pan, G. Cheng, and Y. Tang, "WENC: HTTPS encrypted traffic classification using weighted ensemble learning and Markov chain," in *Proc. IEEE Trustcom/BigDataSE/ICSS*, 2017, pp. 50–57.
- [7] S. Bagui, X. Fang, E. Kalaimannan, S. C. Bagui, and J. Sheehan, "Comparison of machine-learning algorithms for classification of VPN network traffic flow using time-related features," *J. Cyber Security Technol.*, vol. 1, no. 2, pp. 108–126, 2017.
- [8] W. Wang, M. Zhu, J. Wang, X. Zeng, and Z. Yang, "End-to-end encrypted traffic classification with one-dimensional convolution neural networks," in *Proc. IEEE Int. Conf. Intell. Security Informat. (ISI)*, 2017, pp. 43–48.
- [9] H. Yao, C. Liu, P. Zhang, S. Wu, C. Jiang, and S. Yu, "Identification of encrypted traffic through attention mechanism based long short term memory," *IEEE Trans. Big Data*, vol. 8, no. 1, pp. 241–252, Feb. 2022.
- [10] M. Song, J. Ran, and S. Li, "Encrypted traffic classification based on text convolution neural networks," in *Proc. IEEE 7th Int. Conf. Comput. Sci. Netw. Technol. (ICCSNT)*, 2019, pp. 432–436.
- [11] S. Cui, B. Jiang, Z. Cai, Z. Lu, S. Liu, and J. Liu, "A session-packets-based encrypted traffic classification using capsule neural networks," in *Proc. IEEE 21st Int. Conf. High Perform. Comput. Commun. IEEE 17th Int. Conf. Smart City IEEE 5th Int. Conf. Data Sci. Syst. (HPCC/SmartCity/DSS)*, 2019, pp. 429–436.
- [12] Y. Zhang, S. Zhao, J. Zhang, X. Ma, and F. Huang, "STNN: A novel TLS/SSL encrypted traffic classification system based on stereo transform neural network," in *Proc. IEEE 25th Int. Conf. Parallel Distrib. Syst. (ICPADS)*, 2019, pp. 907–910.
- [13] Z. Chen, G. Cheng, B. Jiang, S. Tang, S. Guo, and Y. Zhou, "Length matters: Fast Internet encrypted traffic service classification based on multi-PDU lengths," in *Proc. 16th Int. Conf. Mobil. Sens. Netw. (MSN)*, 2020, pp. 531–538.
- [14] Z. Chen, G. Cheng, Z. Xu, S. Guo, Y. Zhou, and Y. Zhao, "Length matters: Scalable fast encrypted Internet traffic service classification based on multiple protocol data unit length sequence with composite deep learning," *Digit. Commun. Netw.*, vol. 8, no. 3, pp. 289–302, 2022. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2352864821000699>
- [15] G. Baldini, "Analysis of encrypted traffic with time-based features and time frequency analysis," in *Proc. Global Internet Things Summit (GIoTS)*, 2020, pp. 1–5.
- [16] C. Liu, L. He, G. Xiong, Z. Cao, and Z. Li, "FS-Net: A flow sequence network for encrypted traffic classification," in *Proc. IEEE Conf. Comput. Commun.*, 2019, pp. 1171–1179.
- [17] M. Korczynski and A. Duda, "Markov chain fingerprinting to classify encrypted traffic," in *Proc. IEEE Conf. Comput. Commun. (INFOCOM)*, 2014, pp. 781–789.
- [18] J. Zhang, F. Li, F. Ye, and H. Wu, "Autonomous unknown-application filtering and labeling for dl-based traffic classifier update," in *Proc. IEEE Conf. Comput. Commun.*, 2020, pp. 397–405.
- [19] R. Mateless, H. Zlatokrilov, L. Orevi, M. Segal, and R. Moskovitch, "IPvest: Clustering the IP traffic of network entities hidden behind a single IP address using machine learning," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 3, pp. 3647–3661, Sep. 2021.
- [20] G. Aceto, D. Ciunzo, A. Montieri, A. Nascita, and A. Pescapé, "Encrypted multitask traffic classification via multimodal deep learning," in *Proc. IEEE Int. Conf. Commun.*, 2021, pp. 1–6.
- [21] A. Nascita, A. Montieri, G. Aceto, D. Ciunzo, V. Persico, and A. Pescapé, "XAI meets mobile traffic classification: Understanding and improving multimodal deep learning architectures," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 4, pp. 4225–4246, Dec. 2021.
- [22] T. Shapira and Y. Shavitt, "FlowPic: A generic representation for encrypted traffic classification and applications identification," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 2, pp. 1218–1232, Jun. 2021.
- [23] M. Shen, M. Wei, L. Zhu, and M. Wang, "Classification of encrypted traffic with second-order Markov chains and application attribute bigrams," *IEEE Trans. Inf. Forensics Security*, vol. 12, no. 8, pp. 1830–1843, Aug. 2017.
- [24] C. Liu, Z. Cao, Z. Li, and G. Xiong, "LaFFT: Length-aware FFT based fingerprinting for encrypted network traffic classification," in *Proc. IEEE Symp. Comput. Commun. (ISCC)*, 2018, pp. 1–6.
- [25] C. Liu, Z. Cao, G. Xiong, G. Gou, S.-M. Yiu, and L. He, "MaMPF: Encrypted traffic classification based on multi-attribute Markov probability fingerprints," in *Proc. IEEE/ACM 26th Int. Symp. Qual. Service (IWQoS)*, 2018, pp. 1–10.
- [26] X. Wang, S. Chen, and J. Su, "App-net: A hybrid neural network for encrypted mobile traffic classification," in *Proc. IEEE Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, 2020, pp. 424–429.
- [27] M. Shen, J. Zhang, L. Zhu, K. Xu, and X. Du, "Accurate decentralized application identification via encrypted traffic analysis using graph neural networks," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 2367–2380, 2021.
- [28] S. Xu, G. Geng, X. Jin, D. Liu, and J. Weng, "Seeing traffic paths: Encrypted traffic classification with path signature features," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 2166–2181, 2022.
- [29] X. Ma et al., "Context-aware website fingerprinting over encrypted proxies," in *Proc. IEEE Conf. Comput. Commun.*, 2021, pp. 1–10.
- [30] G. Bovenzi, G. Aceto, D. Ciunzo, V. Persico, and A. Pescapé, "A big data-enabled hierarchical framework for traffic classification," *IEEE Trans. Netw. Sci. Eng.*, vol. 7, no. 4, pp. 2608–2619, Oct.–Dec. 2020.
- [31] J. Zhang, Y. Xiang, Y. Wang, W. Zhou, Y. Xiang, and Y. Guan, "Network traffic classification using correlation information," *IEEE Trans. Parallel Distrib. Syst.*, vol. 24, no. 1, pp. 104–117, Jan. 2013.
- [32] I. Guarino, G. Aceto, D. Ciunzo, A. Montieri, V. Persico, and A. Pescapé, "Contextual counters and multimodal deep learning for activity-level traffic classification of mobile communication apps during COVID-19 pandemic," *Comput. Netw.*, vol. 219, Dec. 2022, Art. no. 109452. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1389128622004868>
- [33] L. Yang, S. Fu, Y. Luo, Y. Wang, and W. Zhao, "A clustering method of encrypted video traffic based on Levenshtein distance," in *Proc. 17th Int. Conf. Mobil. Sens. Netw. (MSN)*, 2021, pp. 1–8.
- [34] Y. Li, Y. Lu, and S. Li, "EZAC: Encrypted zero-day applications classification using CNN and K-means," in *Proc. IEEE 24th Int. Conf. Comput. Support. Cooperat. Work Design (CSCWD)*, 2021, pp. 378–383.
- [35] H. Wu, X. Chen, G. Cheng, X. Hu, and Y. Zhuang, "BCAC: Batch classifier based on agglomerative clustering for traffic classification in a backbone network," in *Proc. IEEE/ACM 29th Int. Symp. Qual. Service (IWQOS)*, 2021, pp. 1–10.
- [36] Z. Wu, Y. Dong, W. Tian, and J. Jin, "Enhanced rough K-means based flow aggregation for QoS mapping in heterogeneous network environments," *IEEE Trans. Netw. Service Manag.*, vol. 17, no. 2, pp. 1197–1210, Jun. 2020.
- [37] Z. Chen, G. Cheng, Z. Xu, K. Xu, Y. Shan, and J. Zhang, "A3C system: One-stop automated encrypted traffic labeled sample collection, construction and correlation in multi-systems," *Appl. Sci.*, vol. 12, no. 22, 2022, Art. no. 11731. [Online]. Available: <https://www.mdpi.com/2076-3417/12/22/11731>
- [38] S. Li, H. Guo, and N. Hopper, "Measuring information leakage in website fingerprinting attacks and defenses," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security, CCS*, Toronto, ON, Canada, 2018, pp. 1977–1992. [Online]. Available: <https://doi.org/10.1145/3243734.3243832>
- [39] T. Huoh, Y. Luo, and T. Zhang, "Encrypted network traffic classification using a geometric learning model," in *Proc. IFIP/IEEE Int. Symp. Integr. Netw. Manag. (IM)*, 2021, pp. 376–383.
- [40] M. Lotfollahi, M. J. Siavoshani, R. S. H. Zade, and M. Saberian, "Deep packet: A novel approach for encrypted traffic classification using deep learning," *Soft Comput.*, vol. 24, no. 3, pp. 1999–2012, 2020.



Zihan Chen (Member, IEEE) received the B.S. degree in software engineering from Central South University in 2017, and the Ph.D. degree in cyber security from Southeast University in 2023, where he is currently working as a Postdoctoral Fellow with the School of Cyber Science and Engineering. His major research interests include cyber security, encrypted traffic classification, encrypted traffic feature engineering, and deep learning. He works as a Reviewer for multiple journals such as IEEE INTERNET OF THINGS JOURNAL.



Zijun Wei (Student Member, IEEE) received the B.S. degree in computer science from Qingdao University, Qingdao, China, in 2021. He is currently pursuing the master's degree with the Department of Cyber Science and Engineering, Southeast University. His major research interests include traffic measurement and analysis.

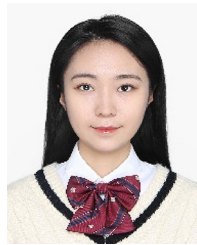


Dandan Niu (Student Member, IEEE) received the B.S. degree in Internet of Things from Jiangnan University in 2021. She is currently pursuing the M.S. degree in cyberspace security from the School of Cyber Science and Engineering, Southeast University. Her major research interests include cyber security and encrypted traffic analysis.



Guang Cheng (Member, IEEE) received the B.S. degree in traffic engineering from Southeast University in 1994, the M.S. degree in computer application from the Hefei University of Technology in 2000, and the Ph.D. degree in computer network from Southeast University, Nanjing, China, in 2003, where he is a Full Professor with the School of Cyber Science and Engineering. He has authored or coauthored seven monographs and more than 100 technical papers, including top journals and top conferences. His research interests include network

security, network measurement, and traffic behavior analysis. He is a Senior Member of CCF.



Nan Fu (Student Member, IEEE) received the M.S. degree from Southeast University in 2022, where she is currently pursuing the Ph.D. degree in cyberspace security from the School of Cyber Science and Engineering. Her major research interests include cyber security, network measurement, and behavior analysis.